

形式语言与自动机

Formal Languages and Automata Theory

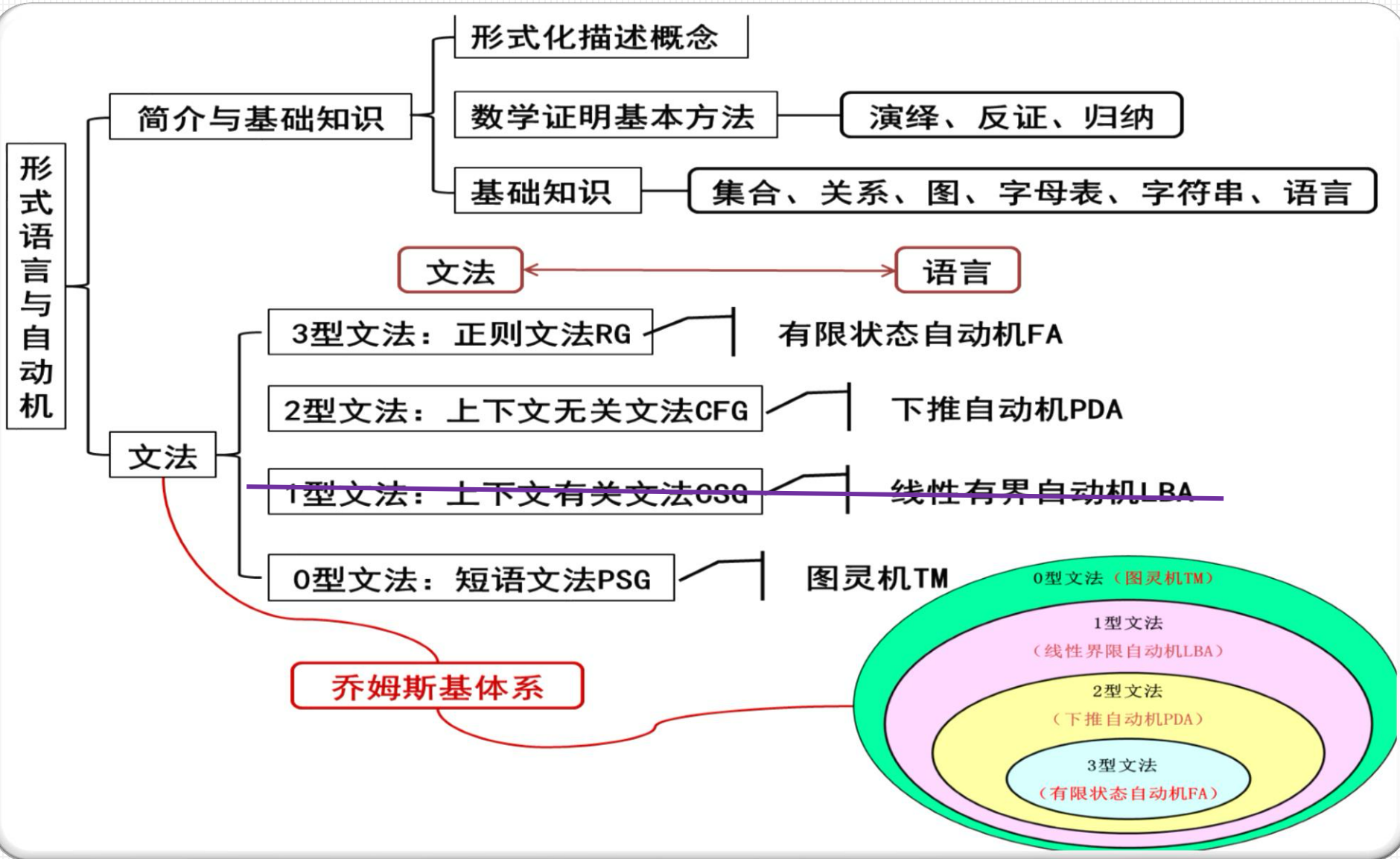
课程总结

计算机科学与技术学院
哈尔滨工业大学（深圳）



郑宜峰

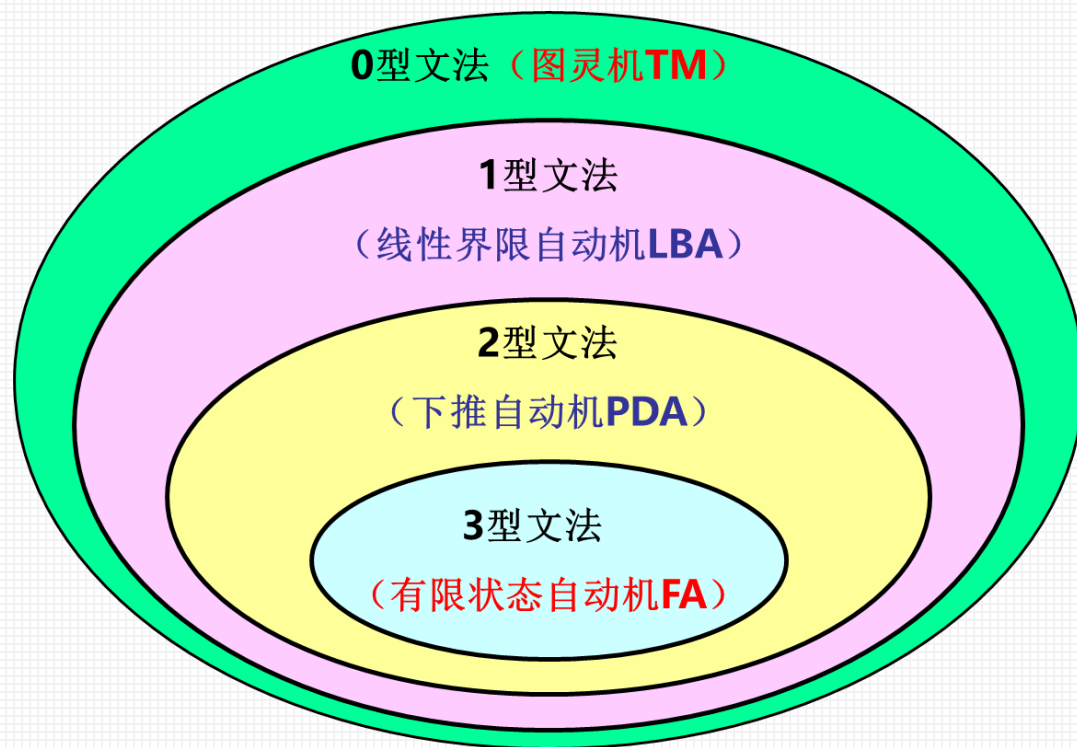
- 正则语言
 - 有穷自动机
 - 正则表达式
 - 正则语言的性质
- 上下文无关语言
 - 上下文无关文法
 - 下推自动机
 - 上下文无关语言的性质
- 图灵机及其扩展



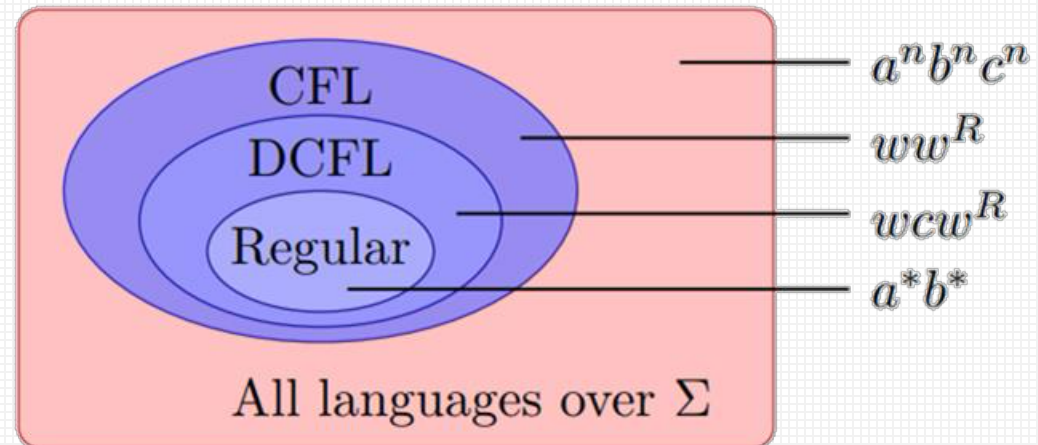
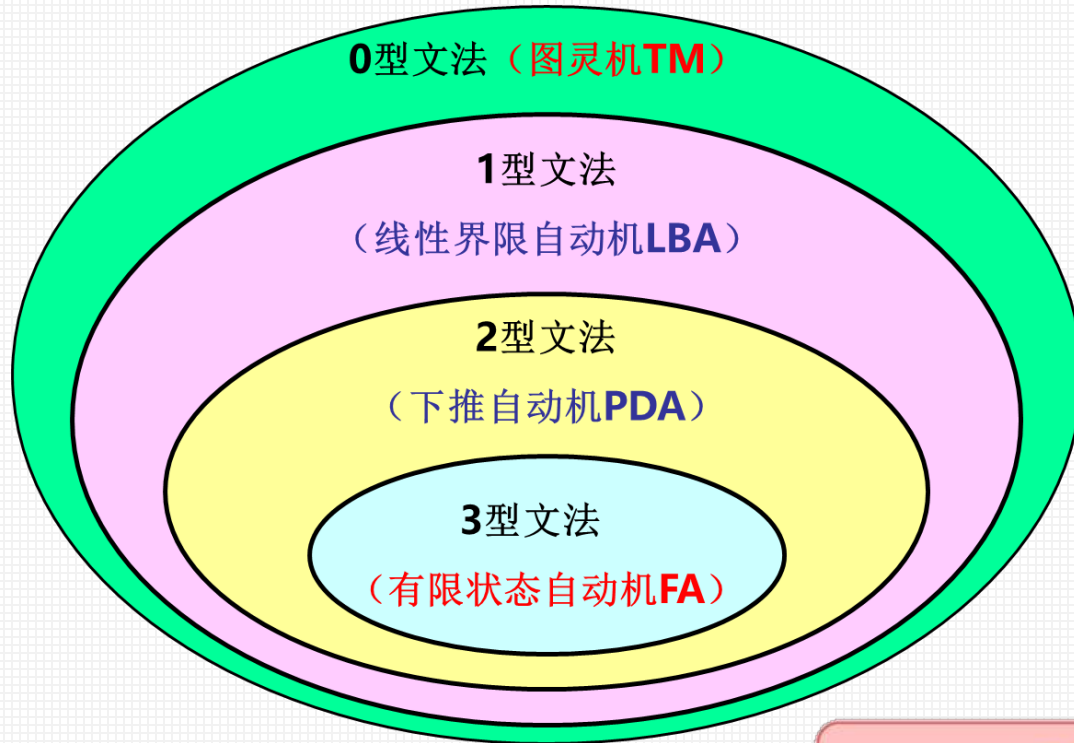
如果文法 $G = (V, T, P, S)$, 符号串 $\alpha \in (V \cup T)^* V (V \cup T)^*, \beta \in (V \cup T)^*$, 产生式都形如

$$\alpha \rightarrow \beta,$$

即每个产生式的左部 α 中至少要有一个变元, 那么:



- (1) 称 G 为0型文法或短语结构文法;
- (2) 若 $|\beta| \geq |\alpha|$, 称 G 为1型文法或上下文有关文法 (Context-Sensitive Grammar);
- (3) 若 $\alpha \in V$, 称 G 为2型文法或上下文无关文法;
- (4) 若 $\alpha \rightarrow \beta$ 都形如 $A \rightarrow aB$ 或 $A \rightarrow a$, 称 G 为3型文法或正则文法。



比较以下三个语言的自动机

$L_1 = \{a^m b^n c^k \mid m, n, k \geq 1\}$ —————→ 正则语言（有穷自动机，正则表达式）

$L_2 = \{a^m b^m c^k \mid m, k \geq 1\}$

$L_3 = \{a^m b^m c^m \mid m \geq 1\}$

—————→ 上下文无关语言（上下文无关文法，下推自动机）

—————→ 图灵机所能识别的语言（图灵机）

比较以下三个语言的自动机

$L_1 = \{a^m b^n c^k \mid m, n, k \geq 1\}$ → 正则语言（有穷自动机，正则表达式）

$L_2 = \{a^m b^m c^k \mid m, k \geq 1\}$

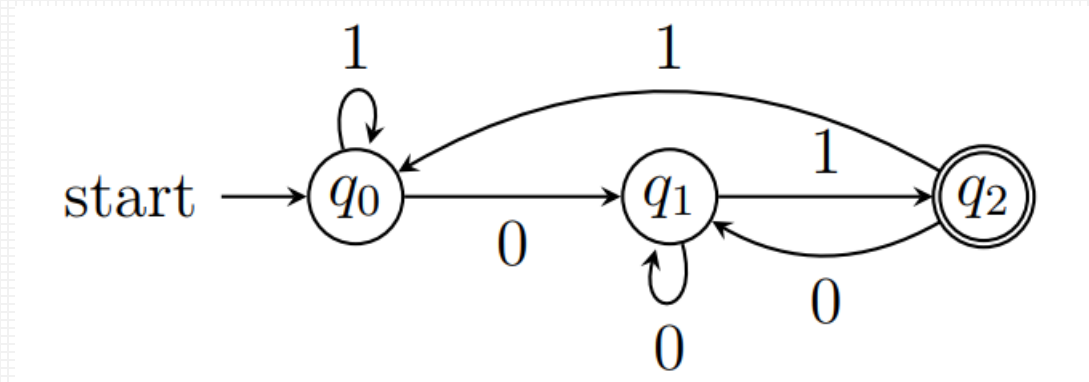
$L_3 = \{a^m b^m c^m \mid m \geq 1\}$

→ 上下文无关语言（上下文无关文法，下推自动机）

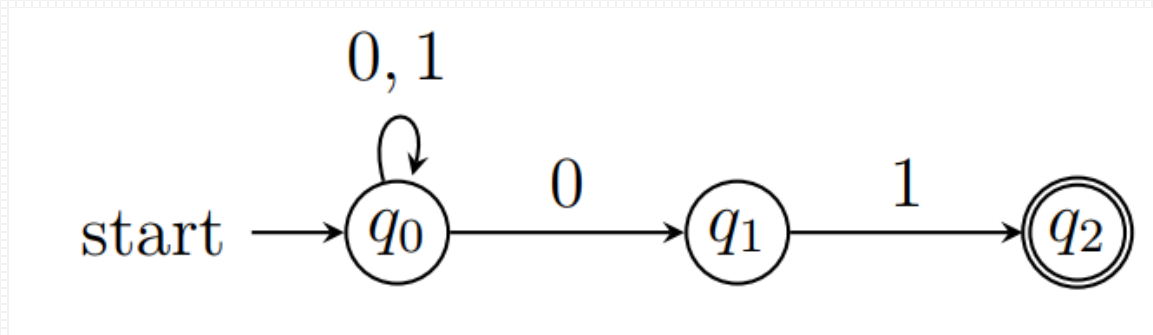
→ 图灵机所能识别的语言（图灵机）

- DFA (确定性的有穷自动机, 每个状态**需要**明确给出对应不同字符输出的转移; 输入一个符号串只会产生**一条**状态转移路径)
- NFA (非确定性的有穷自动机, 每个状态**不需要**明确给出对应不同字符输出的转移; 输入一个符号串只会产生**多条**状态转移路径)
- ε -NFA(非确定性的有穷自动机, 每个状态**不需要**明确给出对应不同字符输出的转移; 当为某个状态**明确定义空转移**时, 则可不消耗输入字符自发进行状态转移。输入一个符号串只会产生**多条**状态转移路径)

- 识别由0和1构成，且以01结尾的符号串的DFA。



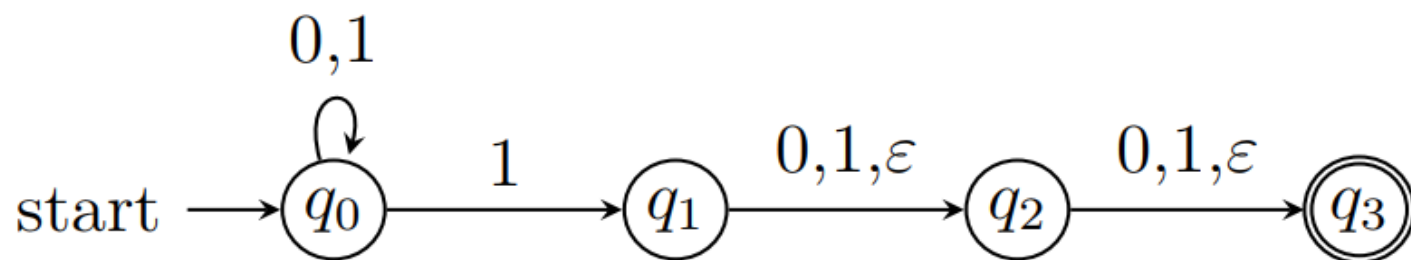
- 识别由0和1构成，且以01结尾的符号串的NFA。



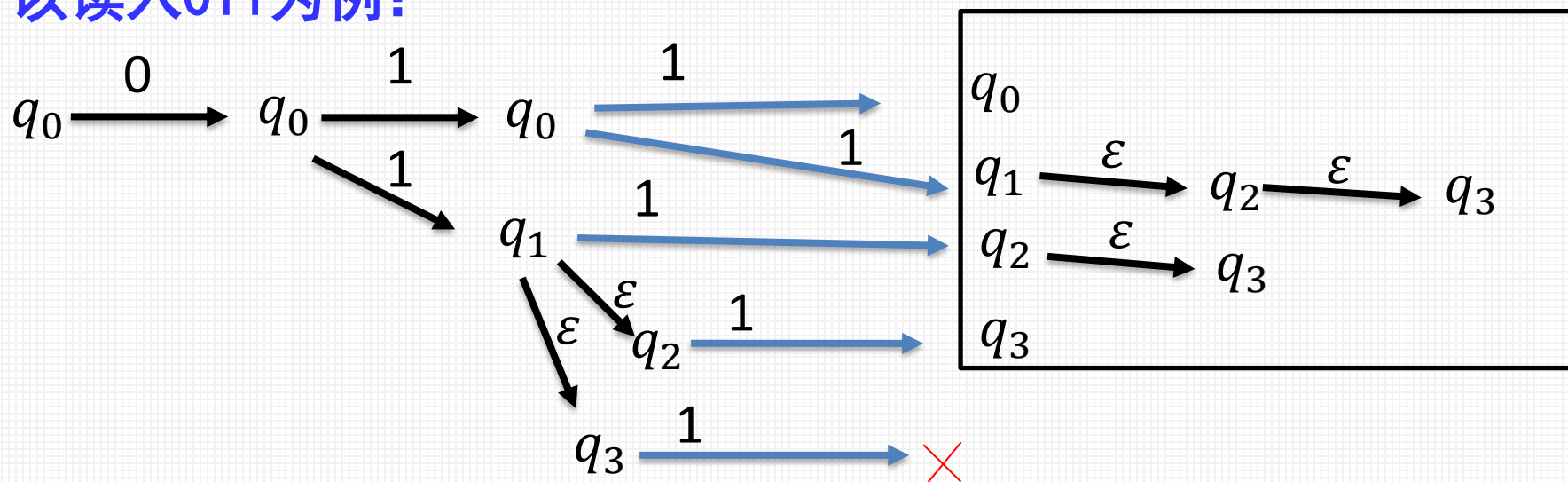
以读入101为例：

开始处于 q_0 , 读入1, 则状态还是 q_0 ;
接下来读入0, 跳转到两个状态, q_0 和 q_1 ;
接下来读入1, 则 $q_0 \rightarrow q_0$, $q_1 \rightarrow q_2$, 即处于 q_0 和 q_2 ;
由于最后的状态中包含有接受状态 q_2 , 该串被接受。

- 识别 $L = \{w \in \{0,1\}^* \mid w \text{ 倒数 3 个字符至少有一个是 } 1\}$ 的 ε -NFA。



以读入011为例：

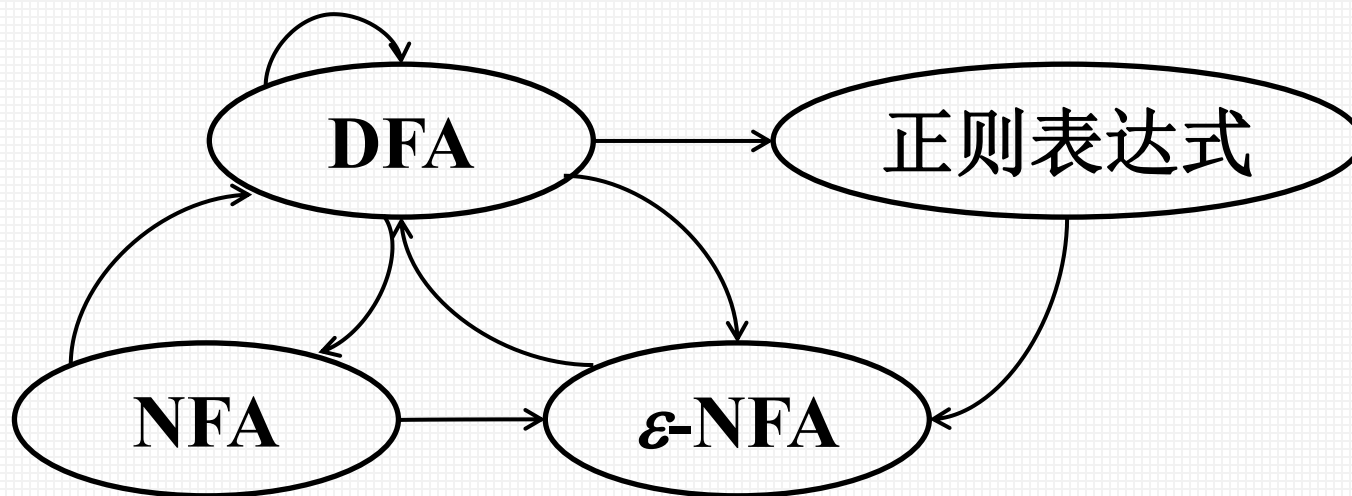


最终所处的状态为 q_0, q_1, q_2, q_3 (011可被接受)。

- 定义：若 $D = (Q, \Sigma, \delta, q_0, F)$ 是一个 DFA, 则 D 接受的语言为
$$L(D) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$
- 定义：若 $N = (Q, \Sigma, \delta, q_0, F)$ 是一个 NFA, 则 N 接受的语言为
$$L(N) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$
- 定义：若 $E = (Q, \Sigma, \delta, q_0, F)$ 是一个 ε -NFA, 则 E 接受的语言为
$$L(E) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

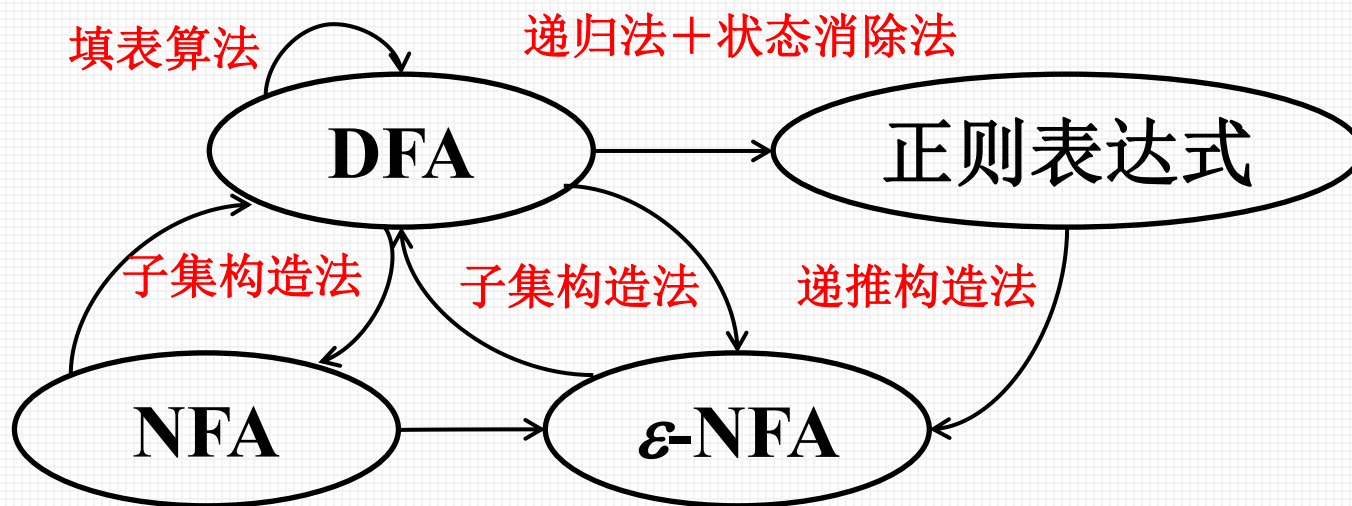
自动机、正则表达式和文法之间的转换

- DFA, NFA, ε -NFA和正则表达式在表示语言的能力上是等价的
 - 相互之间的转换关系



自动机、正则表达式和文法之间的转换

- DFA, NFA, ε -NFA和正则表达式在表示语言的能力上是等价的
 - 相互之间的转换关系



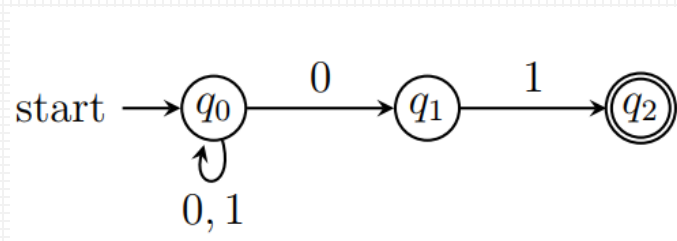
从NFA到DFA：子集构造法

- **目标：** NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ **构造** $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$, 且 $L(D) = L(N)$ 。
- **构造：**
 - 状态集: $Q_D = 2^{Q_N}$, 即 **Q_D 中的每个状态元素是 Q_N 的一个子集**
 - 接受状态集: $F_D = \{S \mid S \subset Q_N, S \cap F_N \neq \emptyset\}$ (set of subsets S of Q_N)
 - F_D 的每一个状态元素亦是 Q_N 的一个子集, 且这些子集与 F_N 的交集不为空。
 - 状态转移函数: $\forall S \subset Q_N, \forall a \in \Sigma, \delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ 。

例如, 假设DFA的某个状态为 $\{q_1, q_2, q_3\}$, 输入符号 a , 那么 $\delta_D(\{q_1, q_2, q_3\}, a)$ 是 $\delta_N(q_i, a)$ 的并集($i = 1, 2, 3$)。

(注意, 这里 $\{q_1, q_2, q_3\}$ 表示DFA的一个状态, 是状态名, 而不是直接看作一个集合。)

- 例：构造接受全部以 01 结尾的串的NFA。



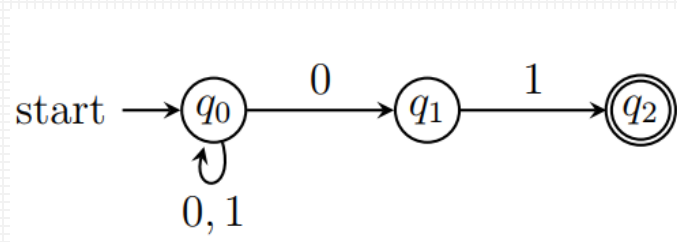
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

1. 列出DFA的所有可能状态集 (为NFA状态集的所有子集)。

	0	1
$\emptyset$		
$\rightarrow \{q_0\}$		
$\{q_1\}$		
$\{q_2\}$		

	0	1
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

- 例：构造接受全部以 01 结尾的串的NFA。



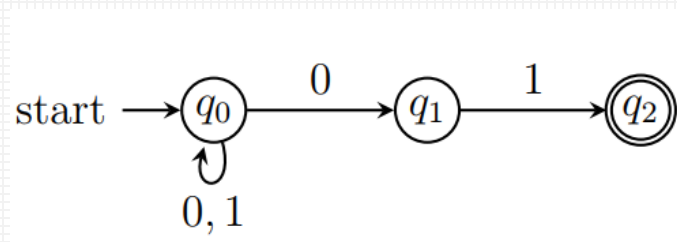
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

2. 转移状态为NFA中相应状态的并集。

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$

	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

- 例：构造接受全部以 01 结尾的串的NFA。



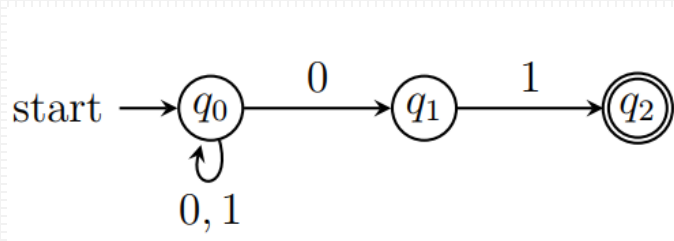
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

3. 删除不能从开始状态达到的状态。

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$

	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

- 例：构造接受全部以 01 结尾的串的NFA。



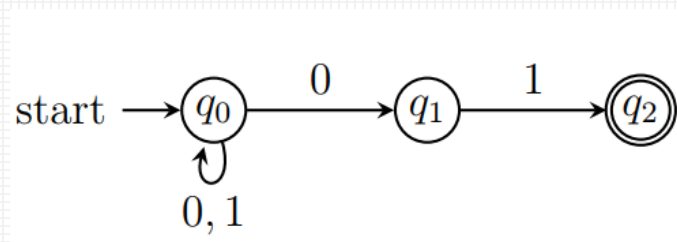
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

4. 含有NFA接受状态的状态集就是DFA的接受状态。

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$

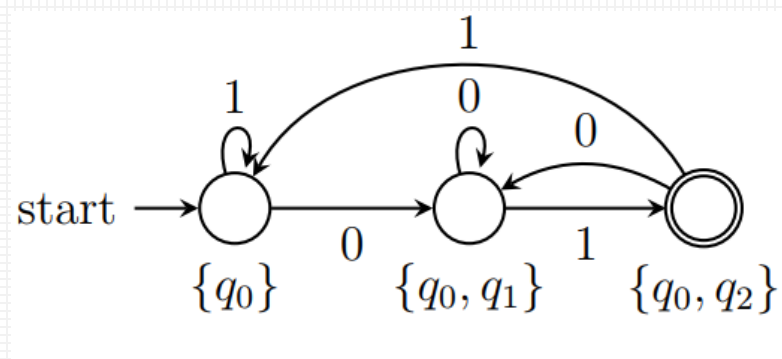
	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

- 例：构造接受全部以 01 结尾的串的NFA。

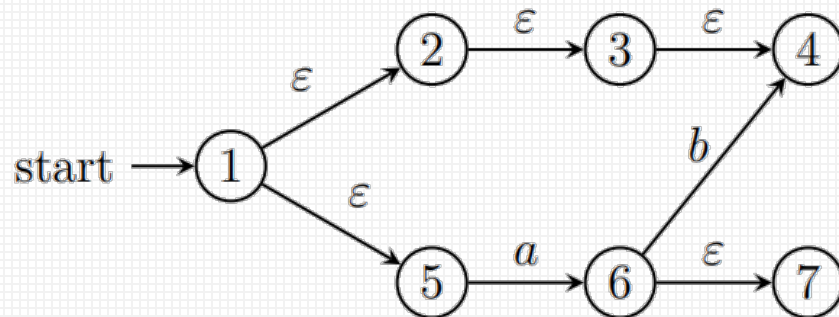


	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

- 含有NFA接受状态的状态集就是DFA的接受状态。



- 定义：状态 q 的 ε -闭包 (ε -closure), 记为 $\text{ECLOSE}(q)$, 表示从 q 经过 $\varepsilon\varepsilon\cdots\varepsilon$ 序列可到达的全部状态的集合。（即经过0个或 >0 个空转移到达的状态集合）
- 递归定义：
 - (1) $q \in \text{ECLOSE}(q)$; (包含当前状态本身)
 - (2) $\forall p \in \text{ECLOSE}(q)$, 若 $r \in \delta(p, \varepsilon)$, 则 $r \in \text{ECLOSE}(q)$



$$\begin{aligned}\text{ECLOSE}(1) &= \{1, 2, 5, 3, 4\} \\ \text{ECLOSE}(2) &= \{2, 3, 4\}\end{aligned}$$

- 子集构造法（消除空转移）：如果 ε -NFA $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ ，构造DFA

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

$$(1) Q_D = 2^{Q_E}$$

$$(2) q_D = \text{ECLOSE}(q_E)$$

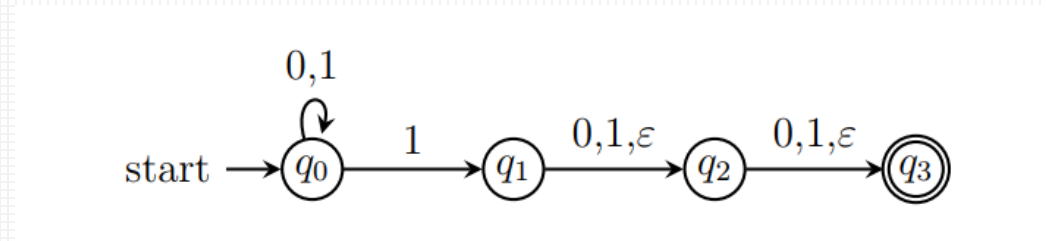
$$(3) F_D = \{S \mid S \subseteq Q_E, S \cap F_E \neq \emptyset\}$$

$$(4) \forall S \subseteq Q_E, \forall a \in \Sigma,$$

$$\delta_D(S, a) = \text{ECLOSE}(\cup_{p \in S} \delta_E(p, a))$$

那么 $L(D) = L(E)$ 。

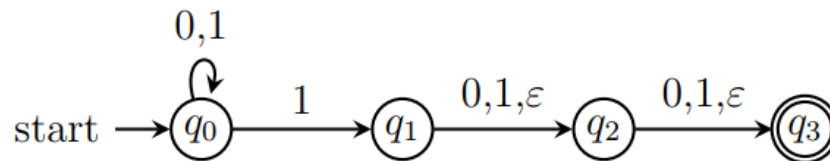
- 例：将下图 L 的 ε -NFA，转为等价的DFA。
 - ε -NFA：全部以01结尾的串



	0	1	ε	ECLOSE()
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_1, q_2, q_3\}$
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_2, q_3\}$
$* q_3$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$

- 子集构造法

- DFA

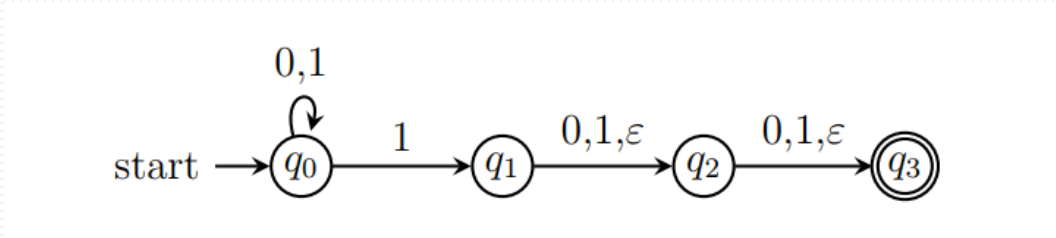


	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
q_1	$\{q_2, q_3\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
$*q_3$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_1, q_2\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$

	0	1
$*\{q_1, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_2, q_3\}$	$\{q_3\}$	$\{q_3\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_1, q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\emptyset$	$\emptyset$	$\emptyset$

- 子集构造法

- DFA

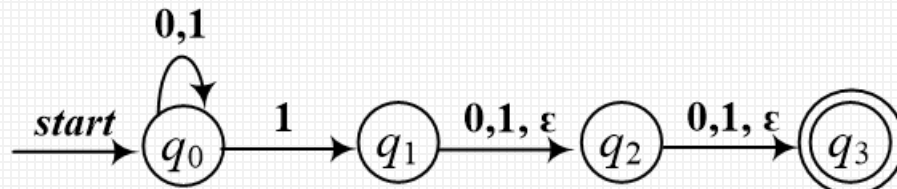


	0	1
→ q_0	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
q_1	$\{q_2, q_3\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
$*q_3$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_1, q_2\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$

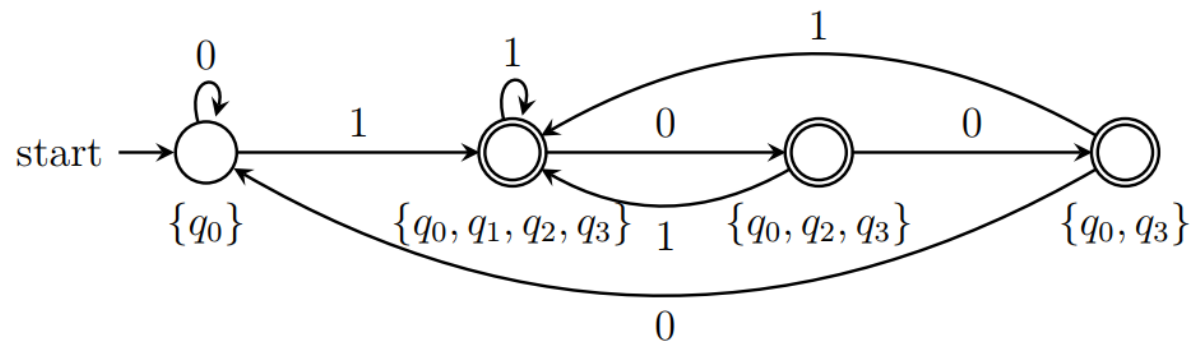
	0	1
$*\{q_1, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_2, q_3\}$	$\{q_3\}$	$\{q_3\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_1, q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\emptyset$	$\emptyset$	$\emptyset$

- 例：将下图 L 的 ε -NFA，转为等价的DFA。

– ε -NFA到DFA



	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$



- 递归法
- 状态消除法

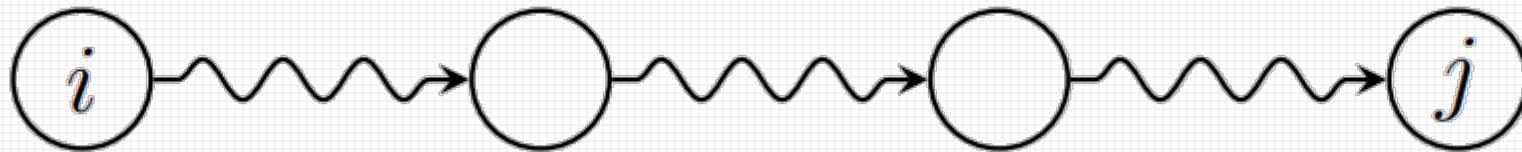
- 递归法
- 状态消除法

- 对 A 的状态进行编号, 令**1**为开始状态, 即

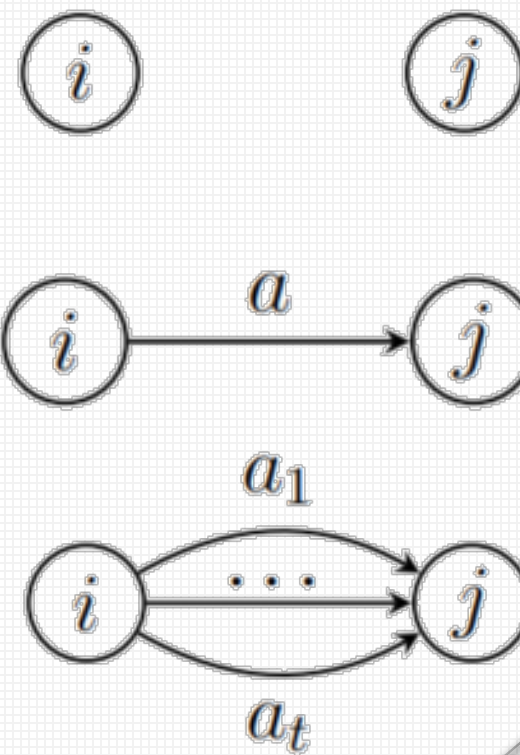
$$A = (\{1, 2, 3, \dots, n\}, \Sigma, \delta, 1, F)$$

设正则表达式 $R_{i,j}^{(k)}$ 表示从 i 到 j 但中间节点状态编号不超过 k 全部路径的字符串集 (简称 k -路径):

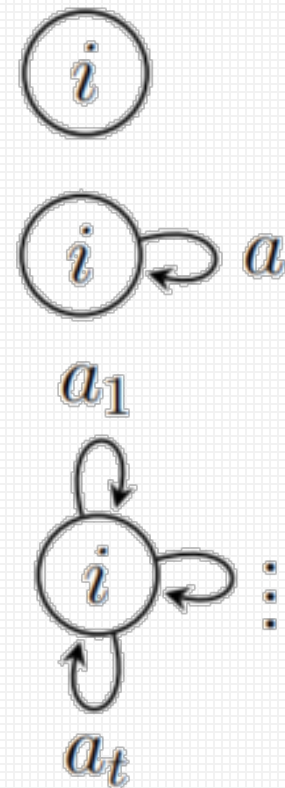
$$R_{i,j}^{(k)} = \{x \mid \hat{\delta}(i, x) = j, x \text{ 经过的状态除两端外都不超过 } k\}$$



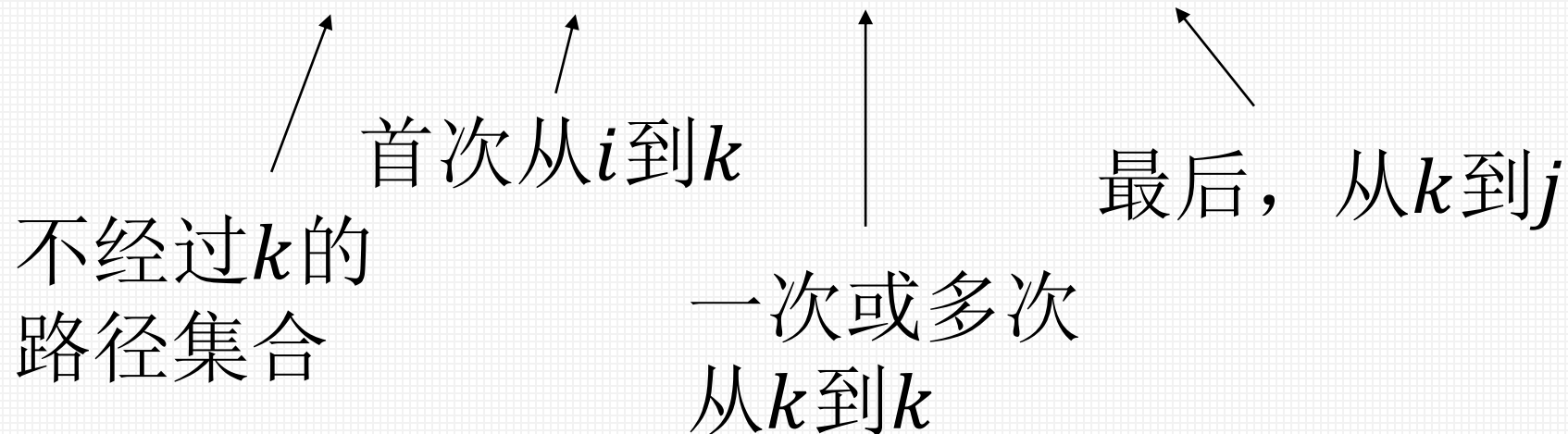
- 归纳基础: $k = 0$, $R_{i,j}^{(0)}$ 为从 i 直接到 j , 没有经过任何中间节点的字符串集合
- 情况1: i 不等于 j
 - 没有 i 到 j 的状态转移 $R_{i,j}^{(0)} = \emptyset$
 - 有一个 i 到 j 的转移 $R_{i,j}^{(0)} = a$
 - 有多个 i 到 j 的转移 $R_{i,j}^{(0)} = a_1 + \dots + a_t$



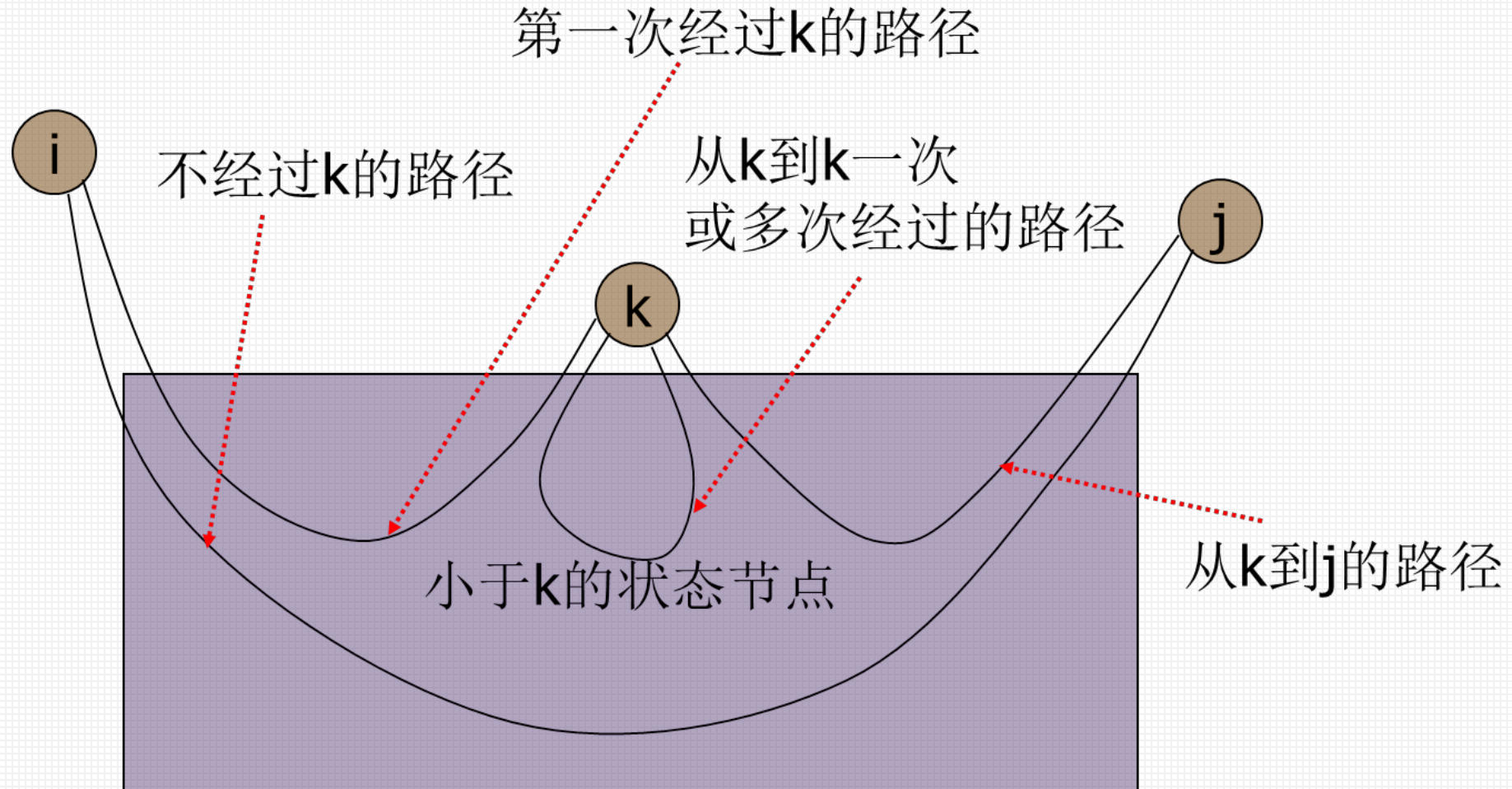
- 归纳基础: $k = 0$, $R_{i,j}^{(0)}$ 为从 i 直接到 j , 没有经过任何中间节点的字符串集合
- 情况2: i 等于 j
 - 状态 i 没有到自身的转移 $R_{i,j}^{(0)} = \varepsilon$
 - 状态 i 有一个到自身的转移 $R_{i,j}^{(0)} = a + \varepsilon$
 - 状态 i 有多个到自身的转移 $R_{i,j}^{(0)} = a_1 + \cdots + a_t + \varepsilon$



- $k > 0$, $R_{i,j}^{(k)}$ 含有两种情况:
 - 不经过状态 k , 此时所经过的状态的最大编号为 $k - 1$
 - 经过状态 k 一次或者多次
- $$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$



- 递归关系: $R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$



- 如果1是开始结点，则和DFA等价的正则表达式就是

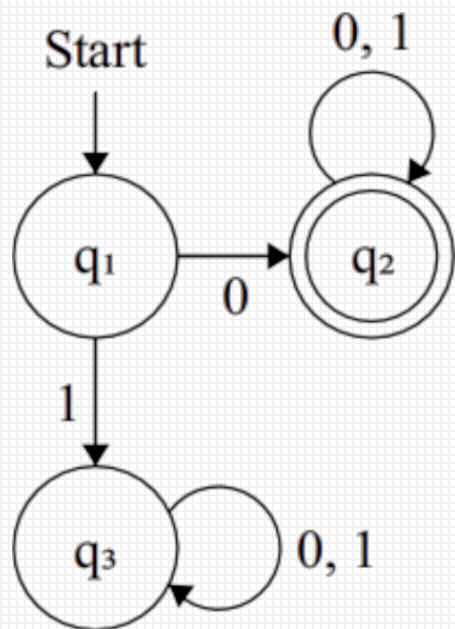
$$\bigcup_{j \in F} R_{1,j}^{(n)}$$

即，从状态1到终结状态的所有路径的集合。

DFA→正则表达式示例

$$R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} (R_{k,k}^{(k-1)})^* R_{k,j}^{(k-1)}$$

- 将以下DFA转化成正则表达式:



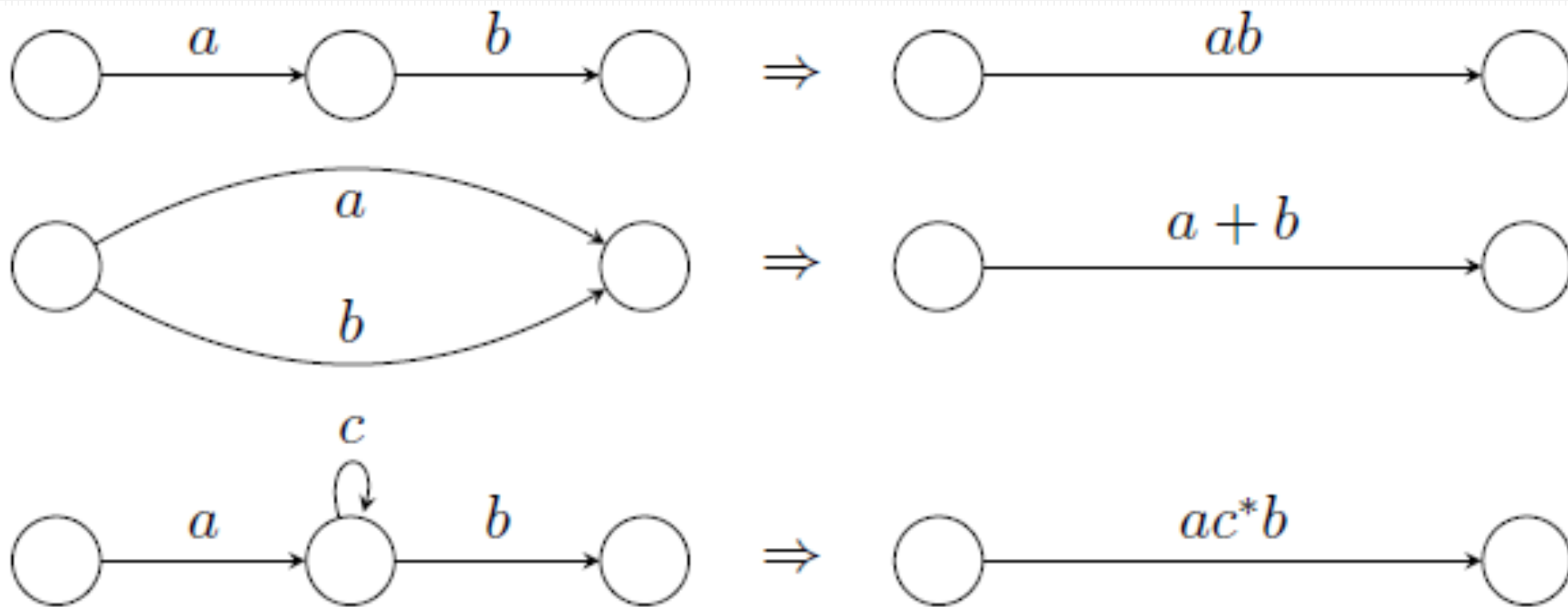
	$k = 0$	$k = 1$	$k = 2$	$k = 3$
R_{11}^k	ϵ	ϵ	ϵ	ϵ
R_{12}^k	0	0	$0(0 + 1)^*$	$0(0 + 1)^*$
R_{13}^k	1	1	1	$1(0 + 1)^*$
R_{21}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{22}^k	$0 + 1 + \epsilon$	$0 + 1 + \epsilon$	$(0 + 1)^*$	$(0 + 1)^*$
R_{23}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{31}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{32}^k	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
R_{33}^k	$0+1+\epsilon$	$0+1+\epsilon$	$0+1+\epsilon$	$(0 + 1)^*$

- 递归法
- 状态消除法

- 从DFA中逐个删除状态
 - 会消除掉状态的原有路径，因此要补上等价的路径
- 用标记了正则表达式的新路径替换被删掉的路径
- 保持“自动机”等价

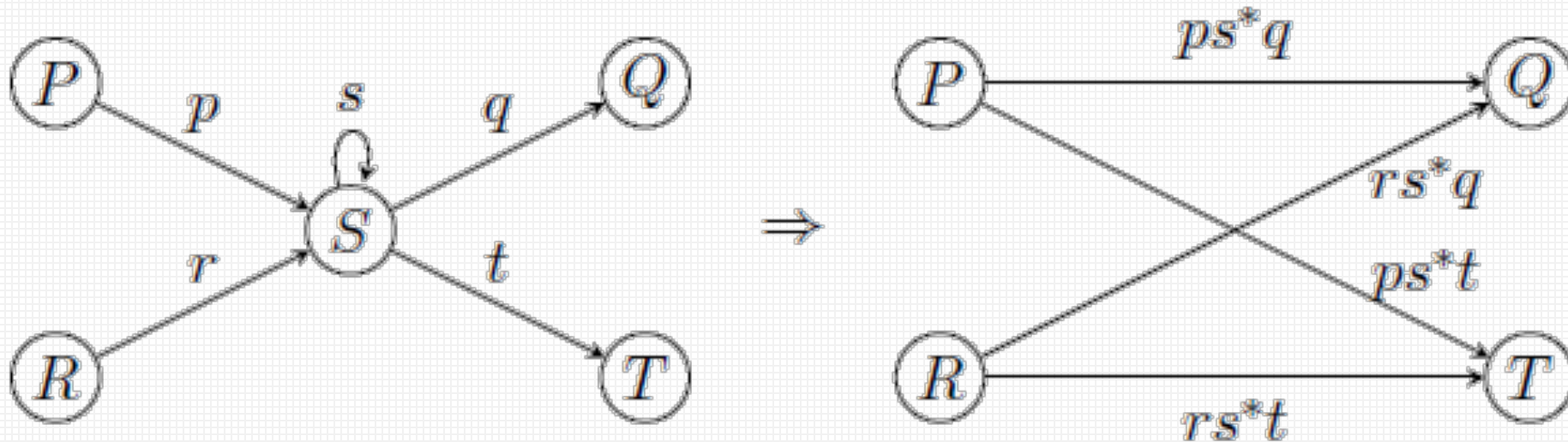
DFA→正则表达式: 状态消除法

- 正则表达式运算（连接，加，闭包）和DFA中路径的关系



DFA→正则表达式: 状态消除法

- 更一般的情况, 要为被删除的状态 S 的每个“入”和“出”路径的组合, 补一条等价的新路径, 并用新的正则表达式表示

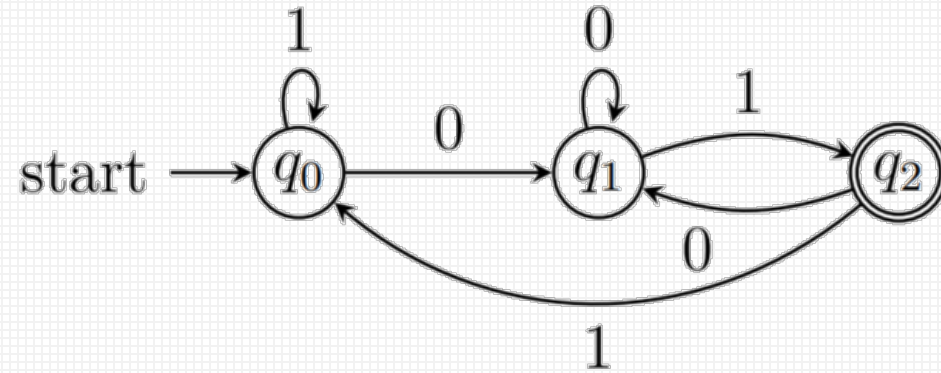


- 2条入, 2条出, 要补4条 (P 到 Q , P 到 T , R 到 Q , R 到 T)
 - 循环的回路另外考虑

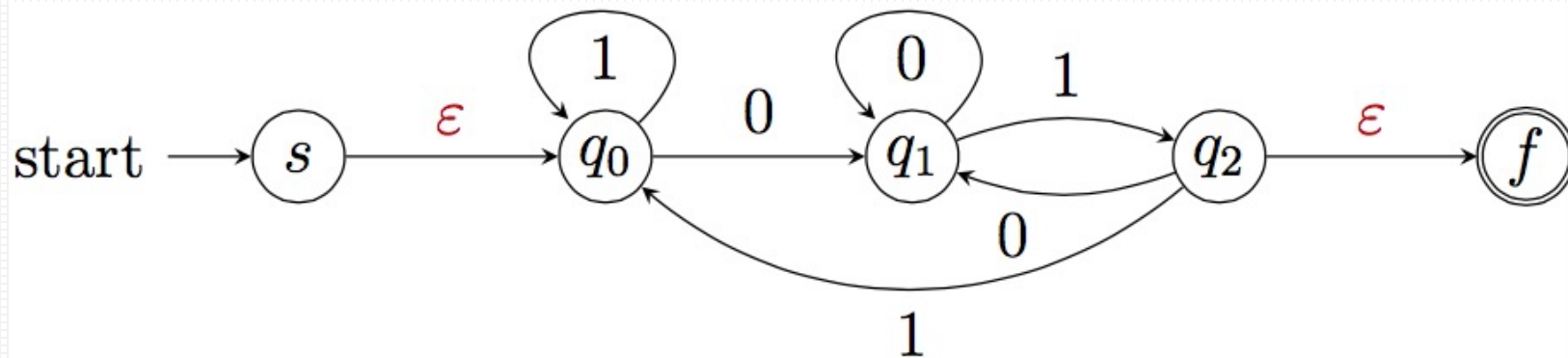
DFA $\rightarrow$ 正则表达式: 状态消除法

- 例: 由0和1构成的串中, 接受全部以01结尾的串。

DFA:

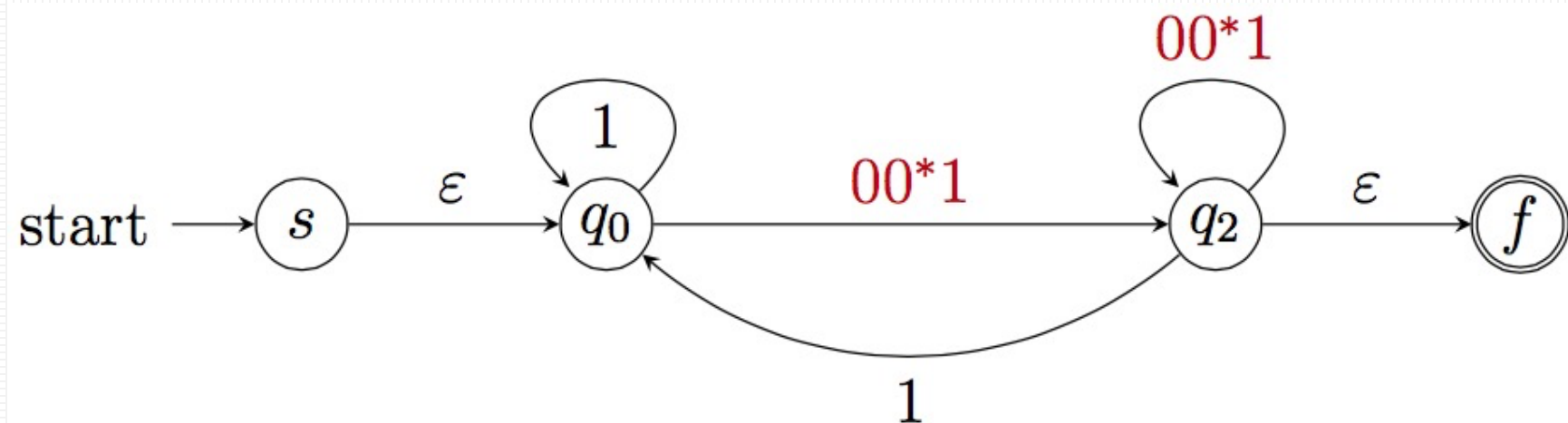
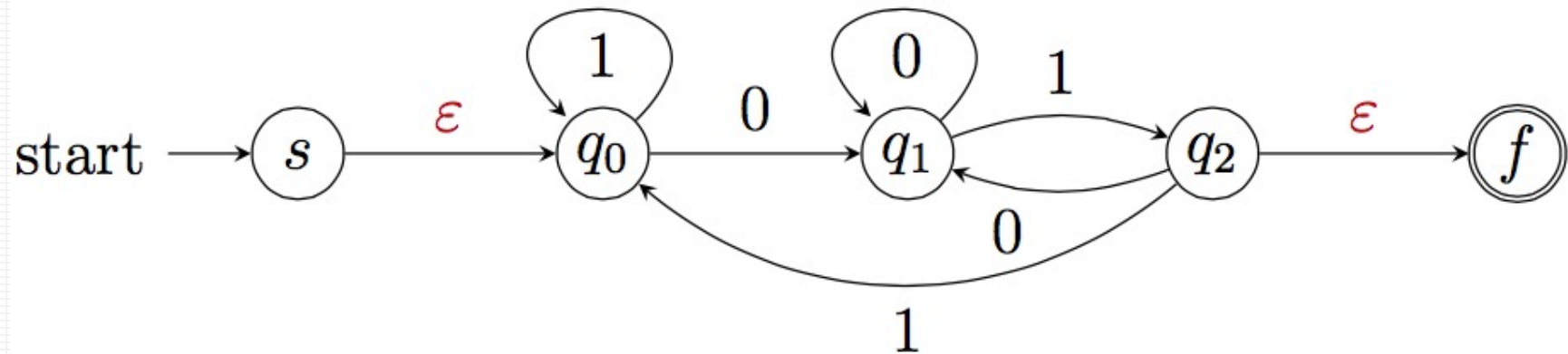


(1) 利用空转移, 增加新的开始 (s) 和结束状态 (f)



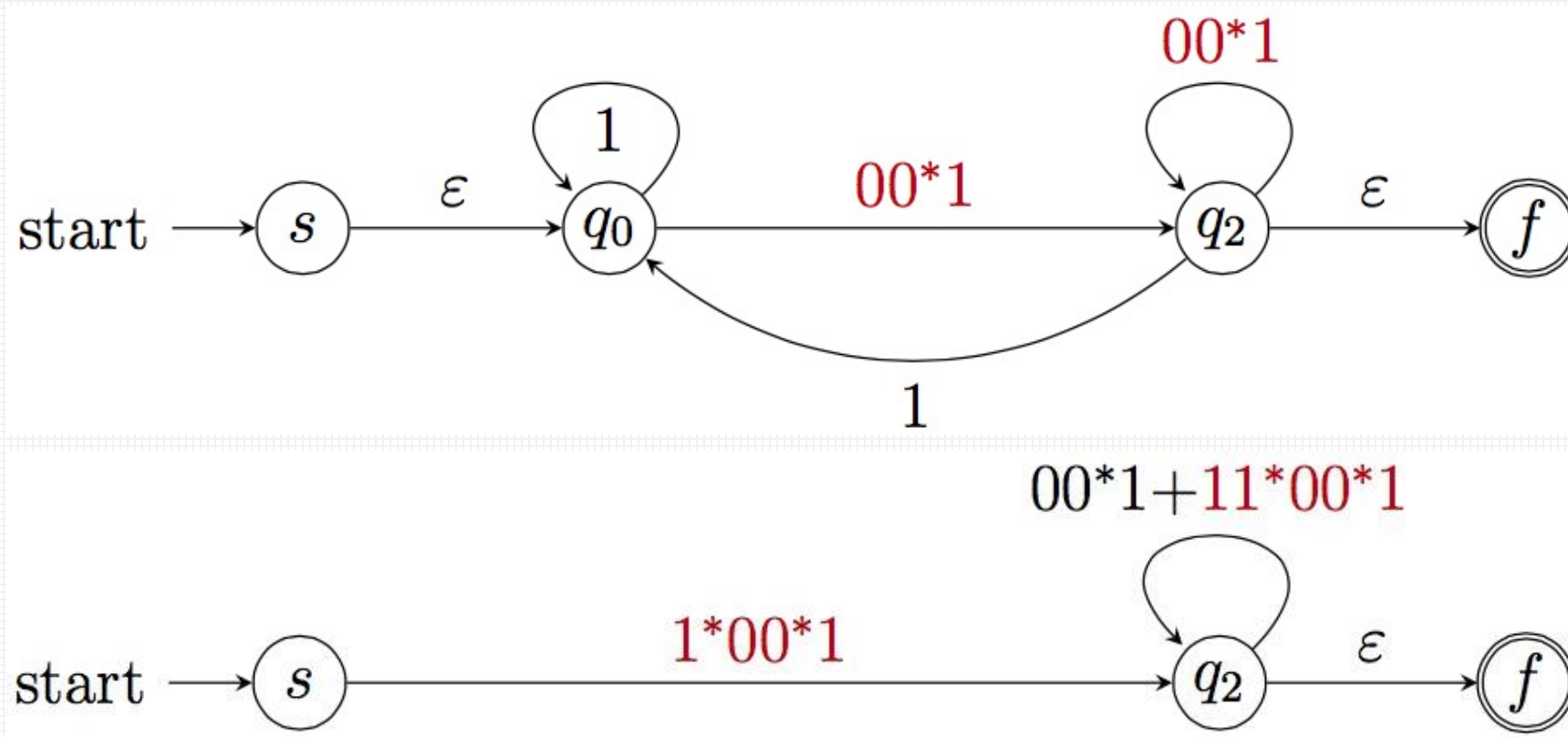
DFA→正则表达式: 状态消除法

(2) 消除状态 q_1 , 需要增加其“入”度 $\{q_0, q_2\}$ 和“出”度 $\{q_2\}$ 之间的边 $q_0 00^* 1 q_2$ 和 $q_2 00^* 1 q_2$, 即 $q_0 \rightarrow q_2$ 和 $q_2 \rightarrow q_2$



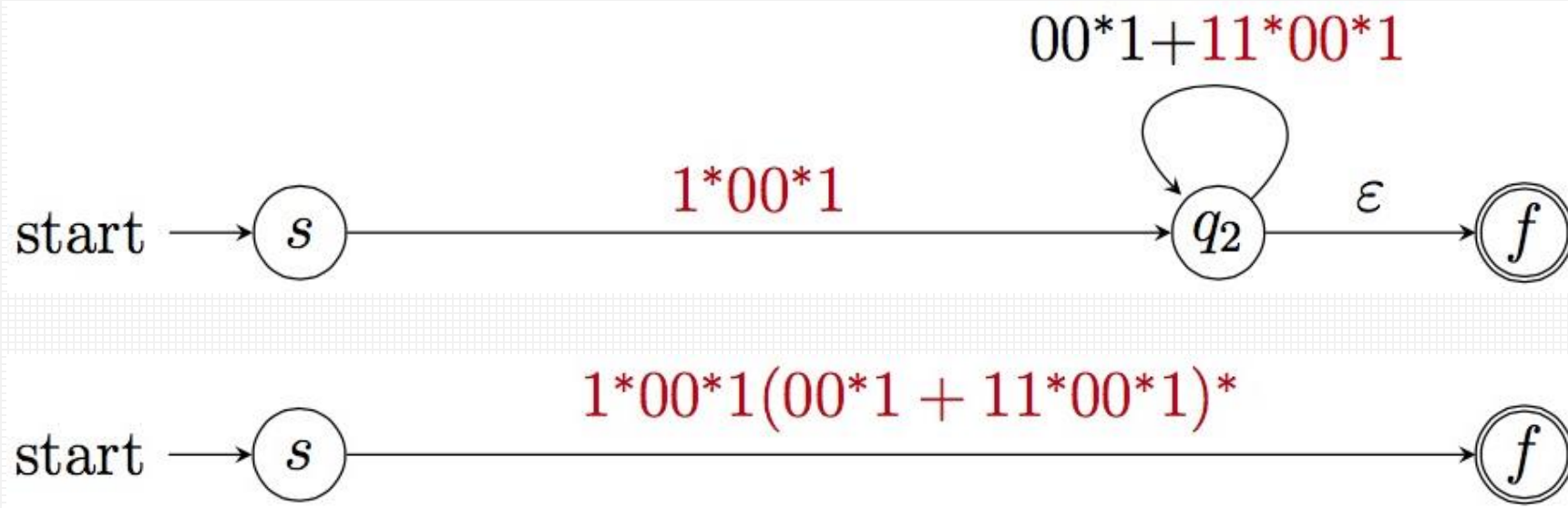
DFA→正则表达式: 状态消除法

(3)消除状态 q_0 , 需要增加其“入”度 $\{s, q_2\}$ 和“出”度 $\{q_2\}$ 之间的边 $s1^*00^*1q_2$ 和 $q_211^*00^*1q_2$, 即 $s \rightarrow q_2$ 和 $q_2 \rightarrow q_2$



DFA→正则表达式: 状态消除法

(4) 消除状态 q_2 , 需要增加其“入”度 $\{s\}$ 和“出”度 $\{f\}$ 之间的边 $s1^*00^*1(00^*1 + 11^*00^*1)^*f$, 即 $s \rightarrow f$

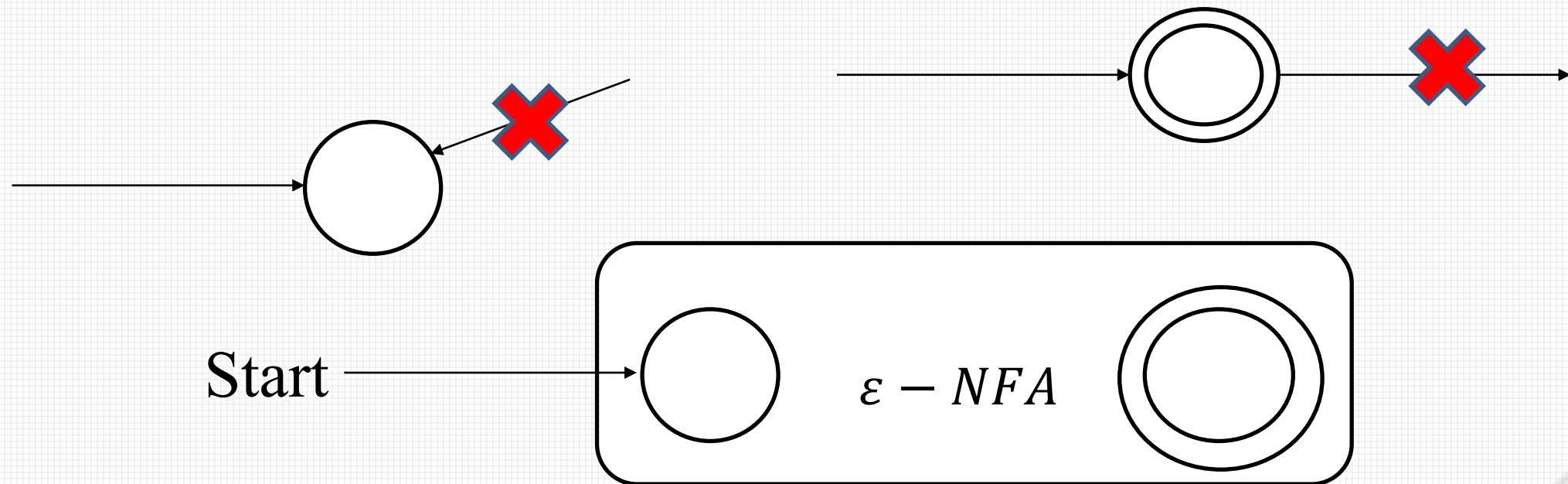


因此, 该自动机的正则表达式为:

$$1^*00^*1(00^*1 + 11^*00^*1)^*$$

正则表达式到自动机：递推构造法

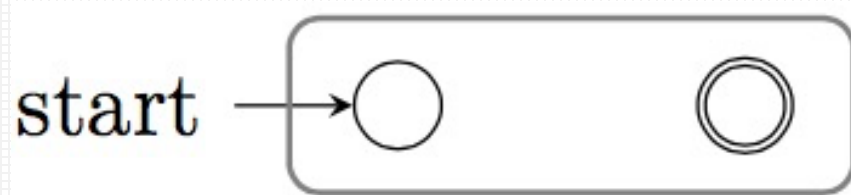
- 定理:每个正则表达式定义的语言，都可被有穷自动机识别。
- $\Leftrightarrow$ 任何正则表达式 R ，都存在与其等价的 ε -NFA A ,即 $\mathbf{L}(A) = \mathbf{L}(R)$ ，并且 A 满足：
 - 仅有一个接受状态；
 - 没有进入开始状态的边；
 - 没有离开接受状态的边。



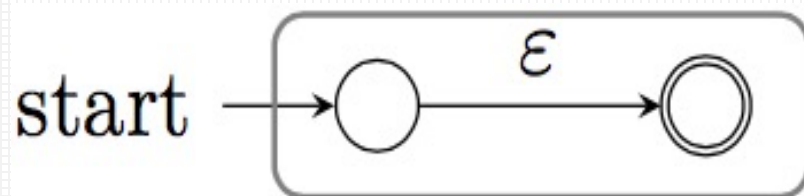
正则表达式到自动机：递推构造法

- 归纳基础：

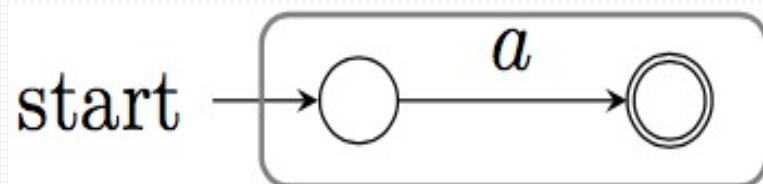
1. 对正则表达式 $\emptyset$ ，有 ε -NFA



2. 对正则表达式 ε ，有 ε -NFA



3. $\forall a \in \Sigma$ ，对正则表达式 a ，有 ε -NFA

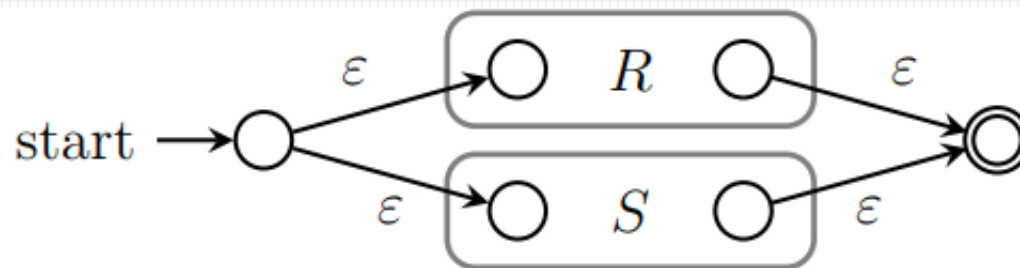


正则表达式到自动机：递推构造法

- 归纳递推:

若 r 和 s 为正则表达式, 则它们对应的 ε -NFA别为 R 和 S 。

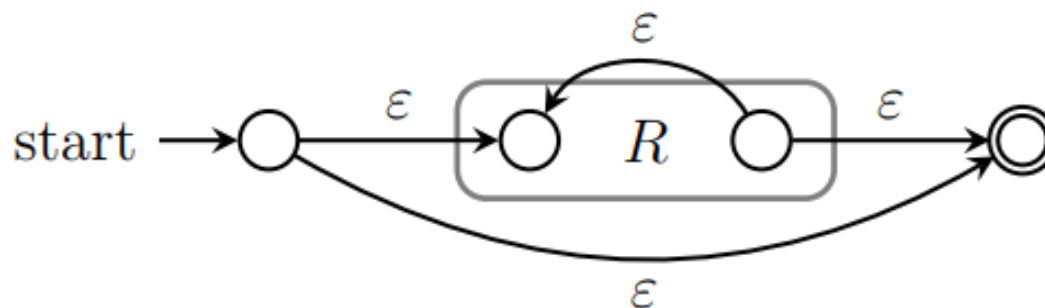
$r + s$ 的 ε -NFA:



rs 的 ε -NFA:



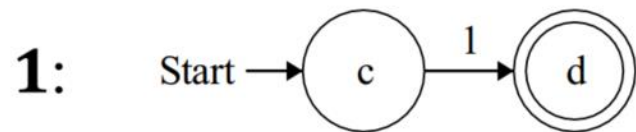
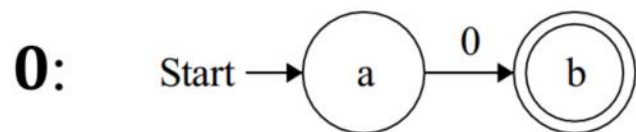
r^* 的 ε -NFA:



因此对于任何正则表达式, 可以构造等价的 ε -NFA。

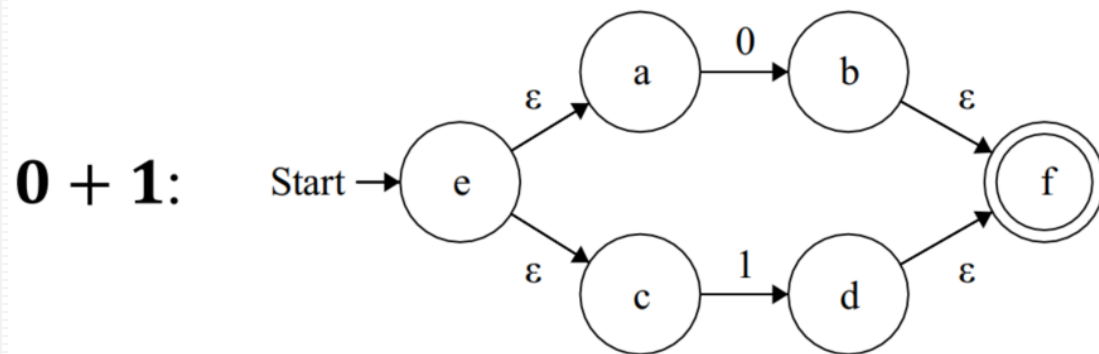
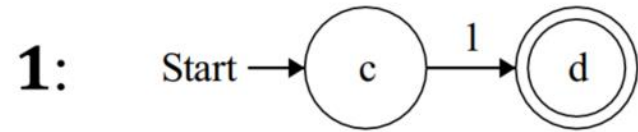
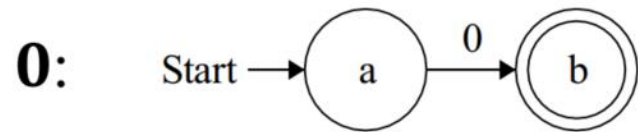
正则表达式到自动机：递推构造法

- 例：将 $0(0+1)^*$ 转化为 ε -NFA。

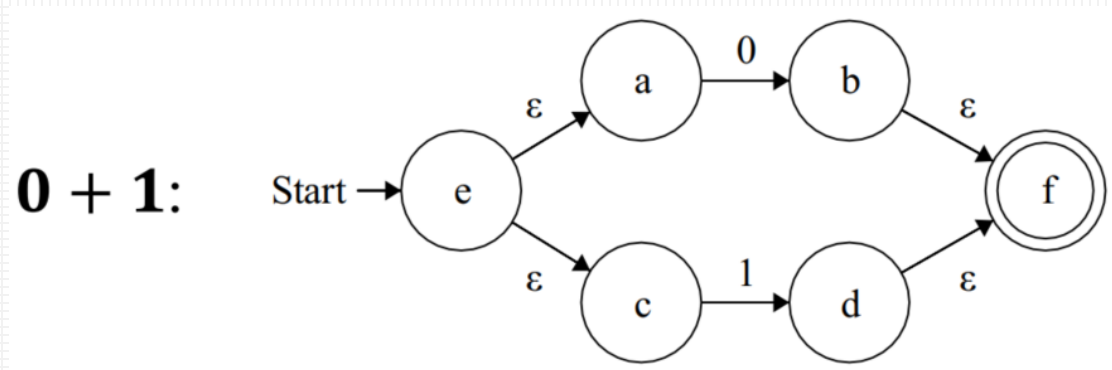


正则表达式到自动机：递推构造法

- 例：将 $0(0+1)^*$ 转化为 ε -NFA。

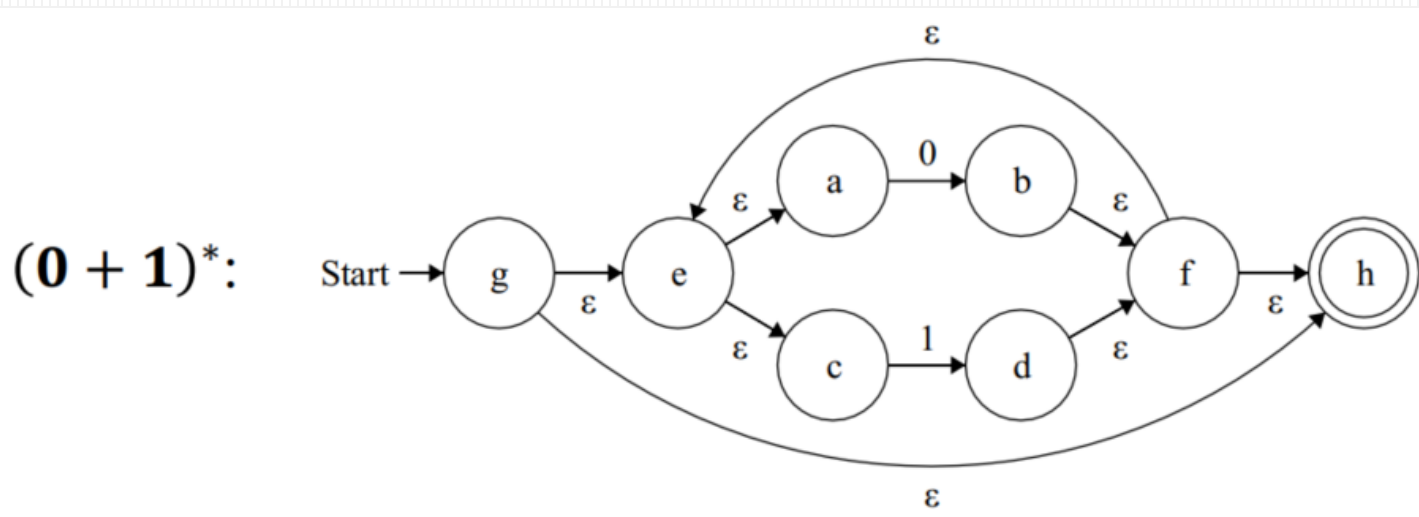
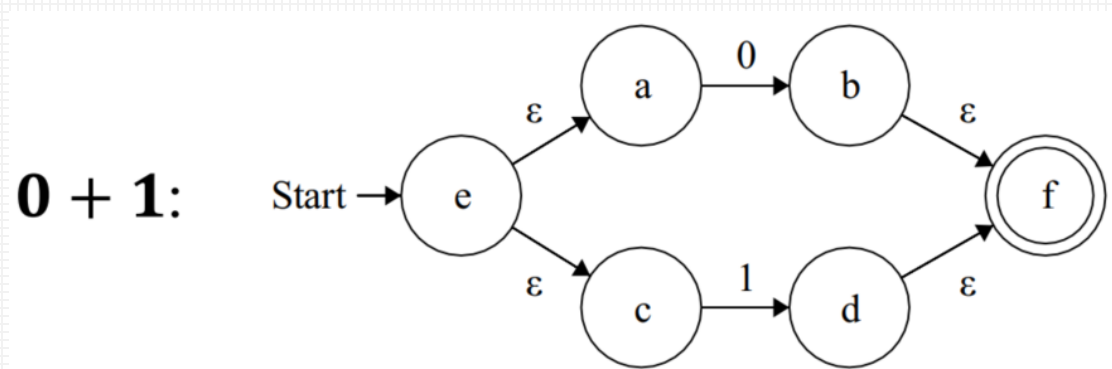


- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



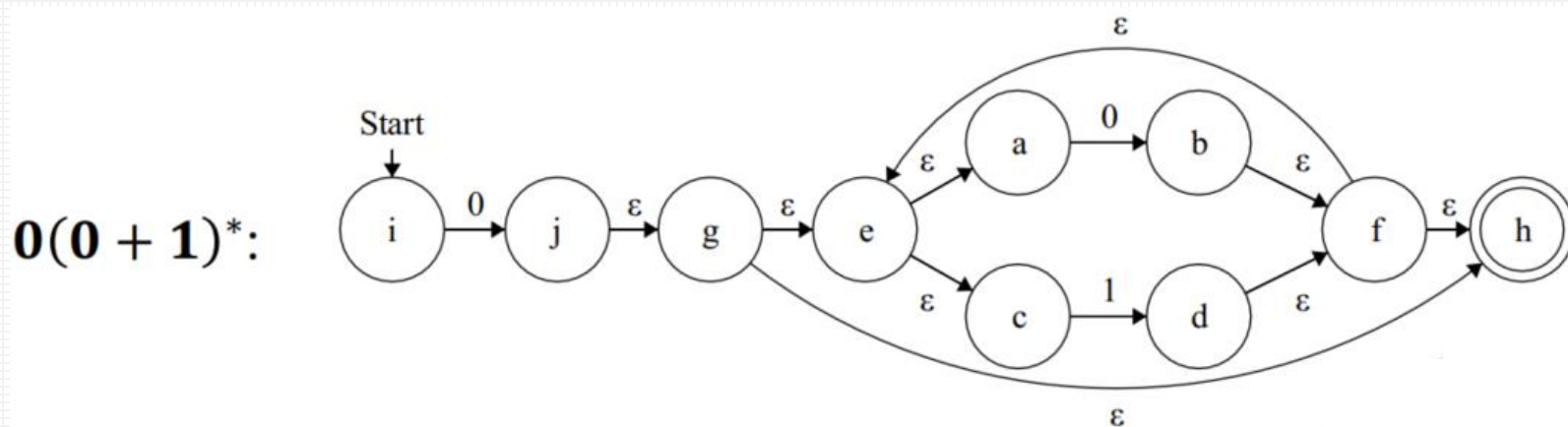
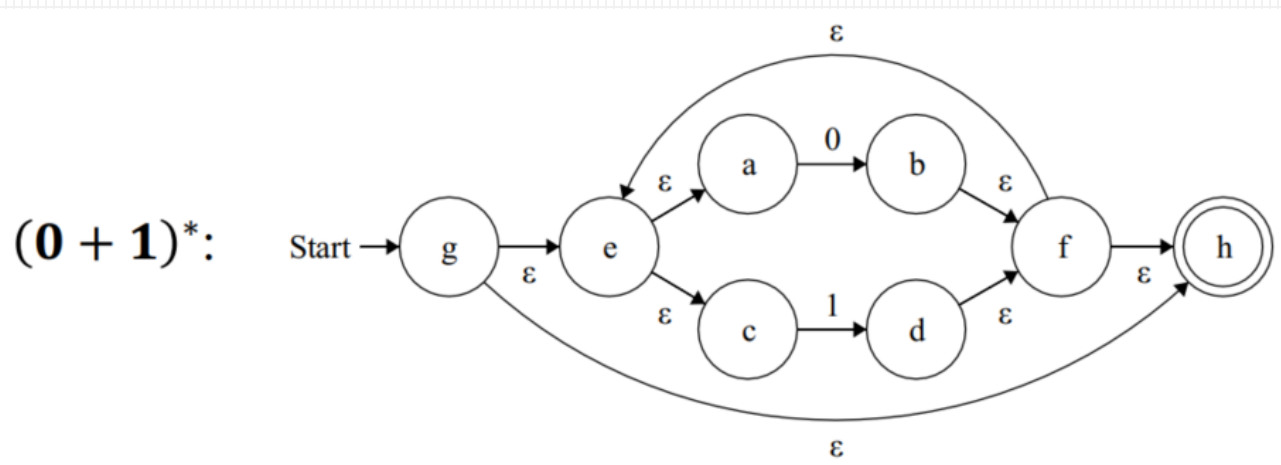
正则表达式到自动机：递推构造法

- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



正则表达式到自动机：递推构造法

- 例：将 $0(0+1)^*$ 转化为 ε -NFA。



- 泵引理(pumping lemma): 如果语言 L 是正则的, 那么存在正整数 N , 对 $w \in L$, 只要 $|w| \geq N$, 就可以将 w 分为三部分 $w = xyz$ 满足:
 - $y \neq \varepsilon$ (或 $|y| > 0$) ;
 - $|xy| \leq N$;
 - $\forall k \geq 0, xy^kz \in L$;
 - 中间的串不论循环几次, 所得的串仍属于该正则语言。

- 泵引理的解释：任何从开始状态到接受状态的路径, 如果长度超过 N , 经过大于 N 个状态, 必定有一个重复状态, 因此会形成一个循环 (loop); 那么, 这个循环可以被重复多次后, 沿原路径还会到达接收状态。
- 泵引理可以用来确定特定语言不是正则语言, 泵引理成立是正则语言判定的必要条件, 但不是充分条件

- 使用泵引理证明某个语言 L 不是正则的
 - 先假设语言 L 是正则的，则一定存在正整数 N ，对 $w \in L(|w| \geq N)$ 满足泵引理。
 - 选取 L 中某个满足长度要求的符号串 w ，分解 $w = xyz$
 - 证明存在某个 $k \geq 0$ ，使得 xy^kz 不属于 L
 - 而由泵引理，如果 L 是正则语言，则 xy^kz 应该属于 L 。
 - 存在矛盾，因此得出结论，语言 L 不是正则的

- 正则语言经某些运算后得到的新语言仍保持正则，称正则语言在这些运算下封闭。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 成员性
- 非空性和无穷性
- 等价性（两个有穷自动机是否等价（接受语言相同））

- 填表算法(table filling)递归寻找DFA中全部的可区分状态对:
 - 基础: 如果 $p \in F$ 而 $q \notin F$, 则 $[p, q]$ 是可区分的;
 - 递推: 如果某个 $a \in \Sigma$, 有 $[r = \delta(p, a), s = \delta(q, a)]$ 是可区分的, 则 $[p, q]$ 是可区分的。

如果填表算法不能区分两个状态, 则这两个状态是等价的.

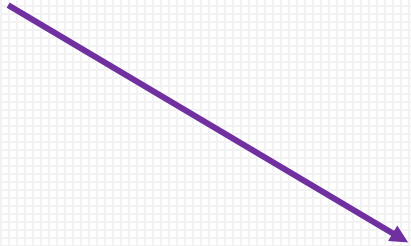
比较以下三个语言的自动机

$L_1 = \{a^m b^n c^k \mid m, n, k \geq 1\}$  正则语言（有穷自动机，正则表达式）

$L_2 = \{a^m b^m c^k \mid m, k \geq 1\}$

$L_3 = \{a^m b^m c^m \mid m \geq 1\}$

 上下文无关语言（上下文无关文法，下推自动机）

 图灵机所能识别的语言（图灵机）

上下文无关法的形式定义

- 定义:上下文无关文法(CFG, Context-Free Grammar, 简称文法) G 是一个四元组

$$G = (V, T, P, S)$$

- 其中,
 - V :变元的有穷集, 变元也称为非终结符或语法范畴
 - T :终结符的有穷集, 且 $V \cap T = \emptyset$ (即字母表), 即语言里面的符号
 - P :产生式的有穷集, 每个产生式包括:
 - 一个变元, 称为产生式的头(head)或左部;
 - 一个产生式符号 $\rightarrow$, 读作定义为;
 - 一个 $(V \cup T)^*$ 中的符号串, 称为体(body)或右部 (即由变元和终结符组成的字符串, 或者是空串)
 - S :初始符号(start symbol) $S \in V$, 表示文法开始的地方 (即如果文法有多个变元, 需要指定一个作为初始符号)

- 产生式 $A \rightarrow \alpha$, 读作 A 定义为 α
- 如果有多个 A 的产生式:

$$A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_n$$

- 可简写为:

$$A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n |$$

- 文法中变元 A 的全体产生式, 称为 A 产生式
- 续例: 回文语言的文法可以表示为

$$G = (\{A\}, \{0,1\}, \{A \rightarrow \varepsilon | 0 | 1 | 0A0 | 1A1\}, A)$$

- 定义:若上下文无关文法 $G = (V, T, P, S)$, 设 $\alpha, \beta, \gamma \in (V \cup T)^*$, $A \in V, A \rightarrow \gamma \in P$, 那么称在 G 中由 $\alpha A \beta$ 可派生出 $\alpha \gamma \beta$, 记为

$$\alpha A \beta \Rightarrow_G \alpha \gamma \beta.$$

- 相应的, 称 $\alpha \gamma \beta$ 可归约为 $\alpha A \beta$.
- 派生 $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$, 即是用产生式 $A \rightarrow \gamma$ 的右部 γ 替换串 $\alpha A \beta$ 中变元 A 得到串 $\alpha \gamma \beta$ 的过程。
- 如果语境中 G 是已知的, 可省略, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$

- 为限制派生的随意性, 要求只替换符号串中最左边变元的派生过程, 称为最左派生, 记为

$$\Rightarrow_{lm}, \Rightarrow_{lm}^*$$

只替换最右的, 称为最右派生, 记为

$$\Rightarrow_{rm}, \Rightarrow_{rm}^*$$

- 任何派生都有等价的最左派生和最右派生。

$$A \Rightarrow^* w \text{ 当且仅当 } A \Rightarrow_{lm}^* w, \text{ 当且仅当 } A \Rightarrow_{rm}^* w。$$

- 定义:上下文无关文法 $G = (V, T, P, S)$ 的语言定义为
$$\mathbf{L}(G) = \{w | w \in T^*, S \xRightarrow{*}_G w\}$$

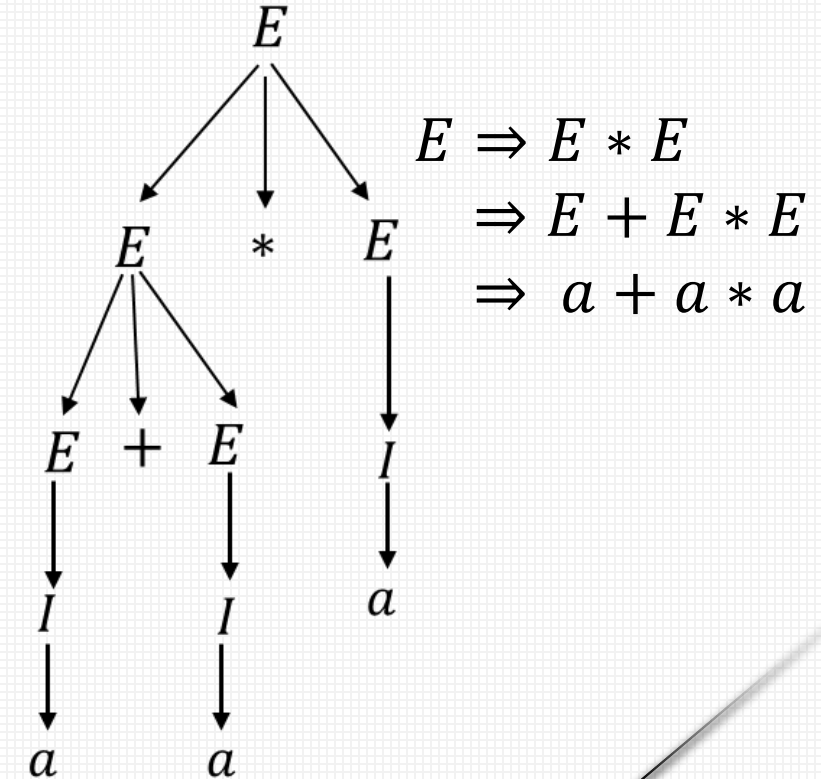
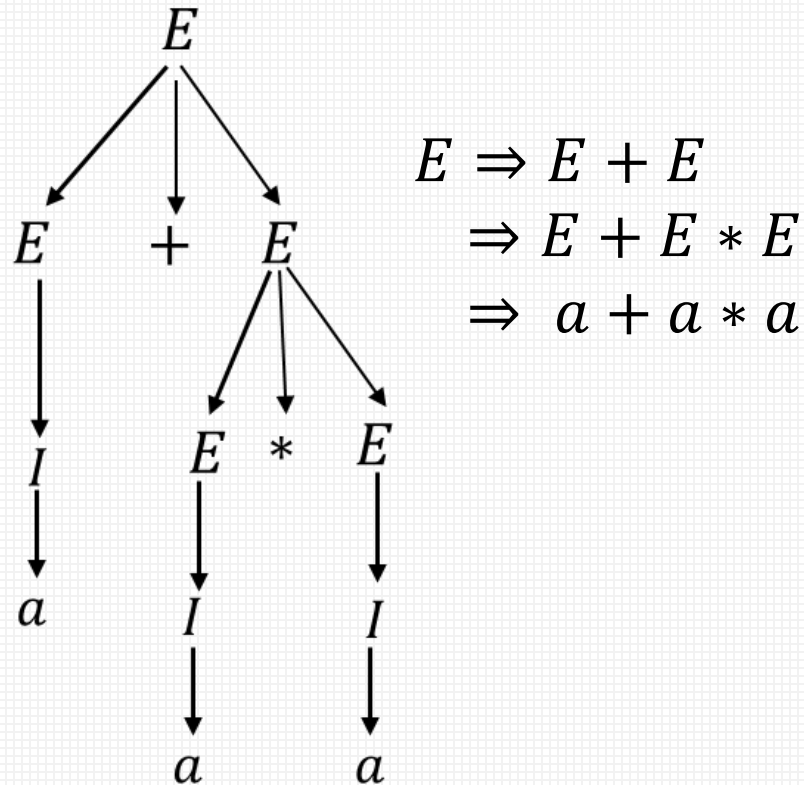
即符号串 w 在 $\mathbf{L}(G)$ 中要满足:

1. w 由终结符组成, 或者是空串;
2. 初始符号 S 能派生出 w .

- 定义:如果CFG G 使某些符号串有两棵不同的语法分析树, 则称该文法是歧义的(ambiguous).

- 例.算数表达式的文法 G_{exp} 中, 对句型 $a + a * a$ 有下面两棵语法树.

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$



- 消除无用符号(useless symbols): 对文法定义语言没有贡献的符号
- 消除 ε -产生式(ε -productions): $A \rightarrow \varepsilon$ (得到语言 $L - \{\varepsilon\}$)
- 消除单元产生式(unit productions): $A \rightarrow B$

- 文法简化步骤的顺序是重要的, 一个可靠的顺序是
 - 消除 ε – 产生式
 - 消除单元产生式
 - 消除非产生的无用符号
 - 消除非可达的无用符号

- 定理: 每个不带 ε 的CFL都可以由这样的CFG G 定义,
 G 中每个产生式的形式都为

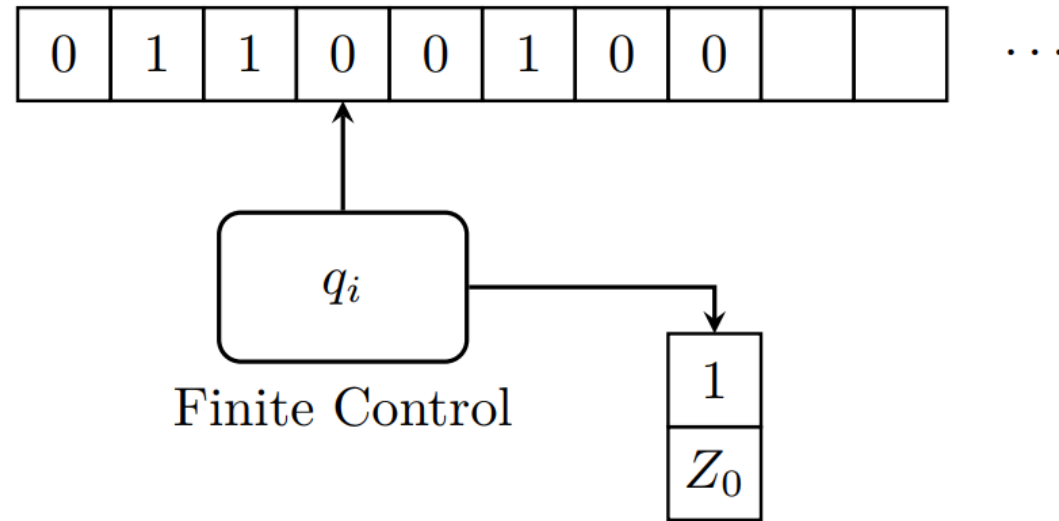
$$A \rightarrow BC \text{ 或 } A \rightarrow a$$

这里的 A, B 和 C 是变元, a 是终结符。

- 设文法不带无用符号， ε -产生式和单元产生式（任何文法都可以化成这种形式）。
- 考虑文法每个形式为 $A \rightarrow X_1X_2 \cdots X_m (m \geq 2)$ 的产生式，若 X_i 为终结符 a ，则引进新变元 C_a 替换 X_i 并增加新产生式 $C_a \rightarrow a$ 。
- 所有产生式的形式变为 $A \rightarrow B_1B_2 \cdots B_m (m \geq 2)$ 和 B_i (即 C_a) $\rightarrow a$ 。
- 对于 $A \rightarrow B_1B_2 \cdots B_m (m > 2)$ ，引入新变元 $D_1, D_2, \dots, D_{m-2}$ ，将其替换为一组级联产生式

$$A \rightarrow B_1D_1, D_1 \rightarrow B_2D_2, \dots, D_{m-2} \rightarrow B_{m-1}B_m$$

即先把 $B_1B_2 \cdots B_m$ 划分为两部分： B_1 和 $D_1: B_2 \cdots B_m$ ；然后 D_1 又可以划分为两部分 B_2 和 $D_2: B_3 \cdots B_m$ ，依次类推。



- 运转：
 - 控制器从输入带读入一个符号 a ，控制栈弹出一个栈顶符号 Z ；
 - 根据符号 a ,符号 Z ,当前所处的状态进行状态的转移并对栈压入0个符号或者压入一个符号串。
 - 压入0个符号即意味着弹出(Pop)操作；
 - 压入一个符号串意味着压入 (Push) 操作。

- PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ 的语言一般通过终态方式接受来定义，记为 $L(P)$ ，定义为

$$L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \gamma), p \in F\}$$

即能够使PDA到达终态的符号串的集合。

- 即需要
 - 1. 检查符号串是否已经消耗完?
 - 2. 检查是否进入了接受状态?

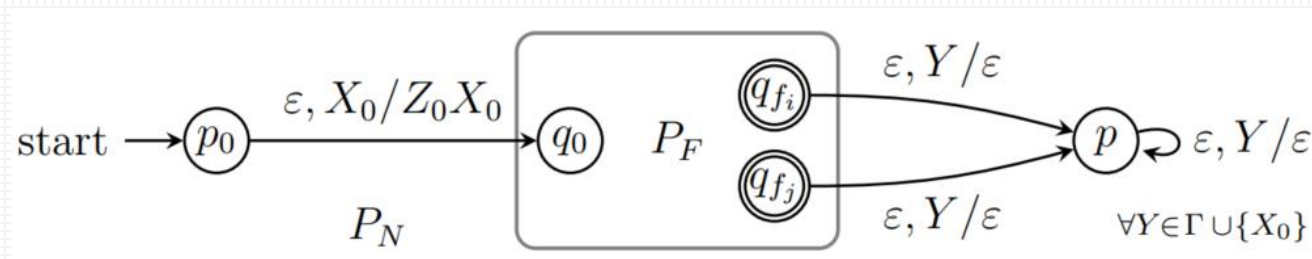
- 另外一种PDA定义的语言是以空栈方式接受的语言, 记为 $N(P)$, 定义为

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \varepsilon)\}$$

即能够使得PDA的栈变空的符号串集合。

- 即需要
 - 1. 检查符号串是否已经消耗完?
 - 2. 检查栈是否已清空?

从终态方式到空栈方式



$P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0, \emptyset)$ 。 δ_N 具体定义：

- P_N 首先将 P_F 的栈底符号 Z_0 压栈, 开始模拟 P_F :

$$\delta_N(p_0, \varepsilon, X_0) = \{(q_0, Z_0 X_0)\};$$

- P_N 模拟 P_F 的动作: $\forall q \in Q, \forall a \in \Sigma \cup \{\varepsilon\}, \forall Y \in \Gamma$:

$$\delta_N(q, a, Y) \text{ 包含 } \delta_F(q, a, Y) \text{ 的全部元素};$$

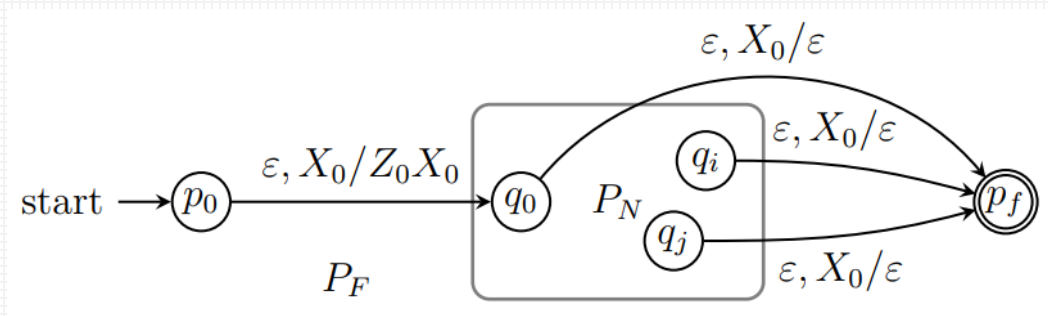
- 从 $q_f \in F$ 开始弹出栈中符号, 即 $\forall q_f \in F, \forall Y \in \Gamma \cup \{X_0\}$:

$$\delta_N(q_f, \varepsilon, Y) \text{ 包含 } (p, \varepsilon);$$

- 在状态 p 时, 弹出全部栈中符号, 即 $\forall Y \in \Gamma \cup \{X_0\}$:

$$\delta_N(p, \varepsilon, Y) = \{(p, \varepsilon)\}。$$

从空栈方式到终态方式



$P_F = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$. δ_F 具体定义:

- P_F 开始时, 将 P_N 栈底符号 Z_0 压栈, 并开始模拟 P_N :

$$\delta_F(p_0, \varepsilon, X_0) = \{(q_0, Z_0 X_0)\};$$

- P_F 模拟 P_N 的动作: $\forall q \in Q, \forall a \in \Sigma \cup \{\varepsilon\}, \forall Y \in \Gamma$:

$$\delta_F(q, a, Y) = \delta_N(q, a, Y);$$

- 在 $\forall q \in Q$ 时, 看到 P_F 的栈底 X_0 , 则转移到新终态 p_f :

$$\delta_F(q, \varepsilon, X_0) = \{(p_f, \varepsilon)\}.$$

由CFG到PDA
由PDA到CFG

由CFG到PDA
由PDA到CFG

- 思路:PDA模拟文法的最左派生, 当输入的符号串消耗完时, 处于按空栈接受的状态或者非接受的状态。
- 步骤:
 - 1. 初始栈底符号是文法的开始符号S。
 - 2. 把产生式的右部压栈, 此时栈顶符号为该右部的第一个符号
 - 3. 如果栈顶符号是最左变元(leftmost variable), 则把它用所有对应的产生式代替 (每个代替可能各自产生一条ID转移路径)
 - 4. 如果栈顶符号是终结符, 且它和要输入的符号匹配, 则弹出该终结符

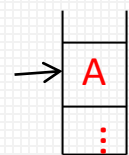
- 给定CFG $G = (V, T, P, S)$, 构造PDA

$$P_N = (\{q\}, T, V \cup T, \delta, q, S, \emptyset),$$

- 其中 δ :

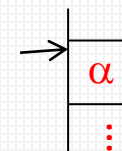
(1) 对每个变元 A : (输入空串, 压入产生式)

变化前:



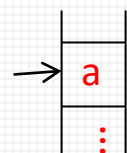
$$\delta(q, \varepsilon, A) = \{(q, \alpha) \mid A \rightarrow \alpha \in P\}$$

变化后:



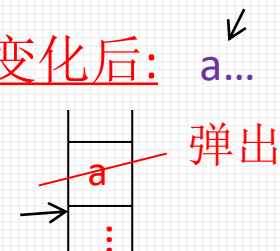
(2) 对每个终结符 a : (输入字符 a , 弹出字符 a)

变化前:



$$\delta(q, a, a) = \{(q, \varepsilon)\}$$

变化后:



- 语言 $L = \{0^n 1^m \mid 1 \leq m \leq n\}$ 。

文法产生式:

$$S \rightarrow AB$$

$$A \rightarrow 0A \mid \varepsilon$$

$$B \rightarrow 0B1 \mid 01$$

PDA 状态转移函数:

$$\delta(q, \varepsilon, S) = \{(q, AB)\}$$

$$\delta(q, \varepsilon, A) = \{(q, 0A), (q, \varepsilon)\}$$

$$\delta(q, \varepsilon, B) = \{(q, 0B1), (q, 01)\}$$

$$\delta(q, 0, 0) = \{(q, \varepsilon)\}, \delta(q, 1, 1) = \{(q, \varepsilon)\}$$

- (1) 对每个变元 A : (输入空串, 压入产生式)

$$\delta(q, \varepsilon, A) = \{(q, \alpha) \mid A \rightarrow \alpha \in P\}$$

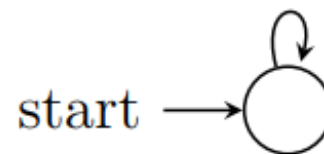
- (2) 对每个终结符 a : (输入字符 a , 弹出字符 a)

$$\delta(q, a, a) = \{(q, \varepsilon)\}$$

$\varepsilon, S/AB$

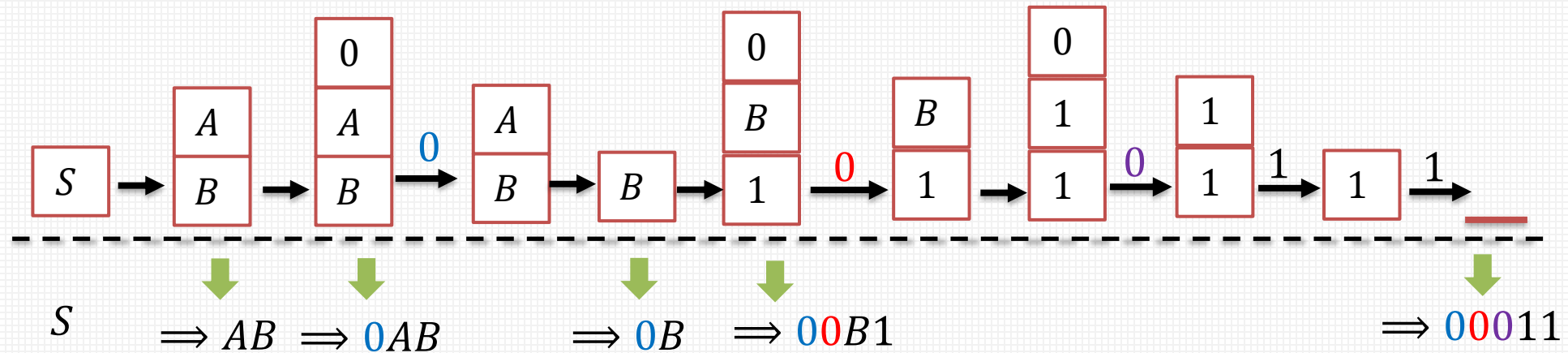
$\varepsilon, A/0A \quad \varepsilon, A/\varepsilon \quad 0, 0/\varepsilon$

$\varepsilon, B/0B1 \quad \varepsilon, B/01 \quad 1, 1/\varepsilon$



- 识别00011为例： $S \Rightarrow AB \Rightarrow 0AB \Rightarrow 0B \Rightarrow 00B1 \Rightarrow 00011$

栈的动作（只针对正确的路径，实际运转中因非确定性会产生其他dead paths）：



$S \rightarrow AB$
 $A \rightarrow 0A | \varepsilon$
 $B \rightarrow 0B1 | 01$

$\delta(q, \varepsilon, S) = \{(q, AB)\}$
 $\delta(q, \varepsilon, A) = \{(q, 0A), (q, \varepsilon)\}$
 $\delta(q, \varepsilon, B) = \{(q, 0B1), (q, 01)\}$
 $\delta(q, 0, 0) = \{(q, \varepsilon)\}, \delta(q, 1, 1) = \{(q, \varepsilon)\}$

由CFG到PDA
由PDA到CFG

- 定理:如果 PDA P_N , 有 $L = N(P)$, 那么 L 是上下文无关语言。
- 设 PDA $P_N = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, 那么构造 CFG $G = (V, \Sigma, P, S)$, 其中 V 和 P 为
- (1) 变元 $V = \{[qXp] \mid p, q \in Q, X \in \Gamma\} \cup \{S\}$; ($[qXp]$ 的意义为 PDA 从状态 q 出发, 处理完(弹出)栈符号 X , 到达状态 p)



这个过程会消耗输入带上某部分的符号串, 也就是该变元能派生出的符号串。

- 构造: 设PDA $P_N = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, 那么构造CFG $G = (V, \Sigma, P, S)$ 。产生式集合 P 包括:
 - (1) 对 $\forall p \in Q$, 构造产生式 $S \rightarrow [q_0 Z_0 p]$;

因为从开始状态 q_0 开始, 完全弹出 Z_0 , 可能到达任意一个状态, 这时候消耗掉的符号串, 也就是这个变元能派生出的符号串, 应该都要能由 S 派生出才行。

- 构造: 设PDA $P_N = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, 那么构造CFG $G = (V, \Sigma, P, S)$ 。产生式集合 P 包括:

(2) 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[qXr_n] \rightarrow a[pY_1r_1][r_1Y_2r_2] \cdots [r_{n-1}Y_nr_n]$, 其中 $a \in \Sigma \cup \{\varepsilon\}$, $X, Y_i \in \Gamma$, $r_1, r_2, \dots, r_n$ 是 Q 中各种可能组合的 n 个状态, 即 r_i 可以是 Q 中任意一个状态。

为了弹出某个栈符号 X , 第一步是利用某个动作将 X 替换为其他栈符号串 $Y_1 Y_2 \dots Y_n$, 然后再一步一步地弹出这些栈符号串。利用这一个动作便可以定义一组产生式。

- 构造: 设PDA $P_N = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, 那么构造CFG $G = (V, \Sigma, P, S)$ 。产生式集合 P 包括:

(2) 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[qXr_n] \rightarrow a[pY_1r_1][r_1Y_2r_2] \cdots [r_{n-1}Y_nr_n]$, 其中 $a \in \Sigma \cup \{\varepsilon\}$, $X, Y_i \in \Gamma$, $r_1, r_2, \dots, r_n$ 是 Q 中各种可能组合的 n 个状态, 即 r_i 可以是 Q 中任意一个状态。

为了弹出某个栈符号 X , 第一步是利用某个动作将 X 替换为其他栈符号串 $Y_1 Y_2 \dots Y_n$, 然后再一步一步地弹出这些栈符号串。利用这一个动作便可以定义一组产生式。

第一步先派生出一个字符 a , 然后剩下的字符串交由变元 $Y_1, Y_2, \dots, Y_n$ 去派生。由于弹出 Y_1 之后到达的那个状态并不确定, 用 r_1 表示。弹完 Y_1 之后, 处于为 r_1 。开始弹 Y_2 , 到达的状态不确定, 定义为 r_2 。依此类推。

- 构造: 设PDA $P_N = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, 那么构造CFG $G = (V, \Sigma, P, S)$ 。产生式集合 P 包括:

(3) 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 若 $Y_1 Y_2 \dots Y_n = \varepsilon$, 则有产生式 $[qXp] \rightarrow a$ 。

由于 $Y_1 Y_2 \dots Y_n = \varepsilon$, 因此一次转移就能将 X 完全弹出, 消耗字符 a , 所以在文法中就是派生 a 。

- 例:将PDA $P = (\{q_0, q_1\}, (0, 1), \{X, Z_0\}, \delta, q_0, Z_0)$ 转为CFG, 其中 δ 如下:

$$(1) \delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$$

$$(2) \delta(q_0, 0, X) = \{(q_0, XX)\}$$

$$(3) \delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$$

$$(4) \delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$$

$$(5) \delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\}$$

$$(6) \delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\}$$

初始符号产生式:

$$S \rightarrow [q_0 Z_0 q_0]$$

$$S \rightarrow [q_0 Z_0 q_1]$$

- 对 $\forall p \in Q$, 构造产生式 $S \rightarrow [q_0 Z_0 p]$;
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[qXr_n] \rightarrow a[pY_1r_1][r_1Y_2r_2] \cdots [r_{n-1}Y_nr_n]$ 。
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 若 $Y_1 Y_2 \dots Y_n = \varepsilon$, 则有产生式 $[qXp] \rightarrow a$ 。

- 例:将PDA $P = (\{q_0, q_1\}, (0, 1), \{X, Z_0\}, \delta, q_0, Z_0)$ 转为CFG, 其中 δ 如下:

$$(1) \delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$$

$$(2) \delta(q_0, 0, X) = \{(q_0, XX)\}$$

$$(3) \delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$$

$$(4) \delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$$

$$(5) \delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\}$$

$$(6) \delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\}$$

$$\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\} \text{ 产生式:}$$

$$[q_0 Z_0 q_0] \rightarrow 0[q_0 X q_0][q_0 Z_0 q_0]$$

$$[q_0 Z_0 q_1] \rightarrow 0[q_0 X q_0][q_0 Z_0 q_1]$$

$$[q_0 Z_0 q_0] \rightarrow 0[q_0 X q_1][q_1 Z_0 q_0]$$

$$[q_0 Z_0 q_1] \rightarrow 0[q_0 X q_1][q_1 Z_0 q_1]$$

-对 $\forall p \in Q$, 构造产生式 $S \rightarrow [q_0 Z_0 p]$;

-对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[q X r_n] \rightarrow a[p Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{n-1} Y_n r_n]$ 。

-对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 若 $Y_1 Y_2 \dots Y_n = \varepsilon$, 则有产生式 $[q X p] \rightarrow a$ 。

由前定义, 因为有两个状态, 故 (r_1, r_2) 共有四种状态组合, 因此有四条产生式。

- 例:将PDA $P = (\{q_0, q_1\}, (0, 1), \{X, Z_0\}, \delta, q_0, Z_0)$ 转为CFG, 其中 δ 如下:

$$(1) \delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$$

$$(2) \delta(q_0, 0, X) = \{(q_0, XX)\}$$

$$(3) \delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$$

$$(4) \delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$$

$$(5) \delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\}$$

$$(6) \delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\}$$

$\delta(q_0, 0, X) = \{(q_0, XX)\}$ 产生式:

$$[q_0 X q_0] \rightarrow 0[q_0 X q_0][q_0 X q_0]$$

$$[q_0 X q_1] \rightarrow 0[q_0 X q_0][q_0 X q_1]$$

$$[q_0 X q_0] \rightarrow 0[q_0 X q_1][q_1 X q_0]$$

$$[q_0 X q_1] \rightarrow 0[q_0 X q_1][q_1 X q_1]$$

- 对 $\forall p \in Q$, 构造产生式 $S \rightarrow [q_0 Z_0 p]$;
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[q X r_n] \rightarrow a[p Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{n-1} Y_n r_n]$ 。
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 若 $Y_1 Y_2 \dots Y_n = \varepsilon$, 则有产生式 $[q X p] \rightarrow a$ 。

- 例: 将PDA $P = (\{q_0, q_1\}, (0, 1), \{X, Z_0\}, \delta, q_0, Z_0)$ 转为CFG, 其中 δ 如下:

$$(1) \delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$$

$$(2) \delta(q_0, 0, X) = \{(q_0, XX)\}$$

$$(3) \delta(q_0, 1, X) = \{(q_1, \varepsilon)\}$$

$$(4) \delta(q_1, 1, X) = \{(q_1, \varepsilon)\}$$

$$(5) \delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\}$$

$$(6) \delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\}$$

$$\delta(q_0, 1, X) = \{(q_1, \varepsilon)\} \text{ 产生式: } [q_0 X q_1] \rightarrow 1$$

$$\delta(q_1, 1, X) = \{(q_1, \varepsilon)\} \text{ 产生式: } [q_1 X q_1] \rightarrow 1$$

$$\delta(q_1, \varepsilon, X) = \{(q_1, \varepsilon)\} \text{ 产生式: } [q_1 X q_1] \rightarrow \varepsilon$$

$$\delta(q_1, \varepsilon, Z_0) = \{(q_1, \varepsilon)\} \text{ 产生式: } [q_1 Z_0 q_1] \rightarrow \varepsilon$$

- 对 $\forall p \in Q$, 构造产生式 $S \rightarrow [q_0 Z_0 p]$;
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 构造 $|Q|^n$ 个产生式 $[q X r_n] \rightarrow a[p Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{n-1} Y_n r_n]$ 。
- 对 $\forall (p, Y_1 Y_2 \dots Y_n) \in \delta(q, a, X)$, 若 $Y_1 Y_2 \dots Y_n = \varepsilon$, 则有产生式 $[q X p] \rightarrow a$ 。

- 定义:如果PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ 满足
(1)对于任何的 (q, a, X) 组合, $q \in Q, a \in \Sigma \cup \{\varepsilon\}, X \in \Gamma, \delta(q, a, X)$ 至多有一个转移;
(2) 如果 $\delta(q, a, X)$ 对于某个 $a \in \Sigma$ 是有进行定义的, 则 $\delta(q, \varepsilon, X) = \emptyset$ 。

则称 P 为确定型下推自动机(DPDA)。

- 在任何情况下都不需要去选择可能的移动就是DPDA。
- 前面学过DFA, 它和NFA识别语言的能力是一样的, 是等价的。但是DPDA与PDA识别语言的能力不一样, 是不等价的。

- 定理:如果 L 是正则语言, 那么存在DPDA P 以终态方式接受 L , 即 $L = \mathbf{L}(P)$ 。
- 定义:如果语言 L 中不存在字符串 x 和 y , 使 x 是 y 的前缀, 称语言 L 满足前缀性质。
- 定理:语言 $L = \mathbf{N}(P)$,即被某个DPDA P 以空栈方式接受, 当且仅当 L 有前缀性质, 且存在以终态方式接受的DPDA P' 使 $L = \mathbf{L}(P')$ 。
- 定理:DPDA P , 语言 $L = \mathbf{L}(P)$, 那么 L 有无歧义的CFG。
- 定理: DPDA P , 语言 $L = \mathbf{N}(P)$, 那么 L 有无歧义的CFG。

- 定理:如果语言 L 是CFL, 那么存在正整数 N , 对 $\forall z \in L$, 只要 $|z| \geq N$, 就可以将 z 分为五部分 $z = uvwxy$ 满足:
 - (1) $vx \neq \varepsilon$ (或 $|vx| > 0$);
 - (2) $|vwx| \leq N$;
 - (3) $\forall i \geq 0, uv^iwx^iy \in L$ 。

- 例:证明 $L = \{0^n 1^n 2^n \mid n \geq 1\}$ 不是上下文无关语言。

(1)假设 L 是CFL, 那么存在整数 N , 对 $\forall z \in L (|z| \geq N)$ 满足泵引理;

(2)从 L 中取 $z = 0^N 1^N 2^N$, 则显然 $z \in L$ 且 $|z| = 3N \geq N$;

(3)由泵引理, z 可被分为 $z = uvwxy$, 且有 $|vwx| \leq N$ 和 $vx \neq \varepsilon$;

(4)那么 vx 可能:

i. 只包含0或1或2, 那么 $uwy \notin L$; (uwy 即删掉部分0, 或部分1, 或部分2, 那么都不会在语言中)

ii. 只包含0和1, 或只包含1和2, 那么也有 $uwy \notin L$; (uwy 即删掉部分0和部分1, 或部分1和部分2, 那么都不会在语言中)

(5)与泵引理 $uwy = uv^0wx^0y \in L$ 矛盾, 假设不成立;

(6) L 不是上下文无关的。

对 $\forall z \in L$, 只要 $|z| \geq N$, 就可以将 z 分为五部分 $z = uvwxy$ 满足:

- (1) $vx \neq \varepsilon$;
- (2) $|vwx| \leq N$;
- (3) $\forall i \geq 0$, $uv^iwx^iy \in L$

- 上下文无关语言运算的封闭性
 - 封闭运算：代换、并、乘、闭包、同态映射、逆同态映射
 - 不封闭运算：交、补
- 定理:若 L 是CFL且 R 是正则语言, 则 $L \cap R$ 是CFL。
 - $L \cap R$ 不是正则语言, 比如考虑 $L = 0^n 1^n, R = 0^* 1^*$, 则 $L \cap R = 0^n 1^n$

- 可判定的性质
 - 空性
 - 无穷性
 - 成员性

比较以下三个语言的自动机

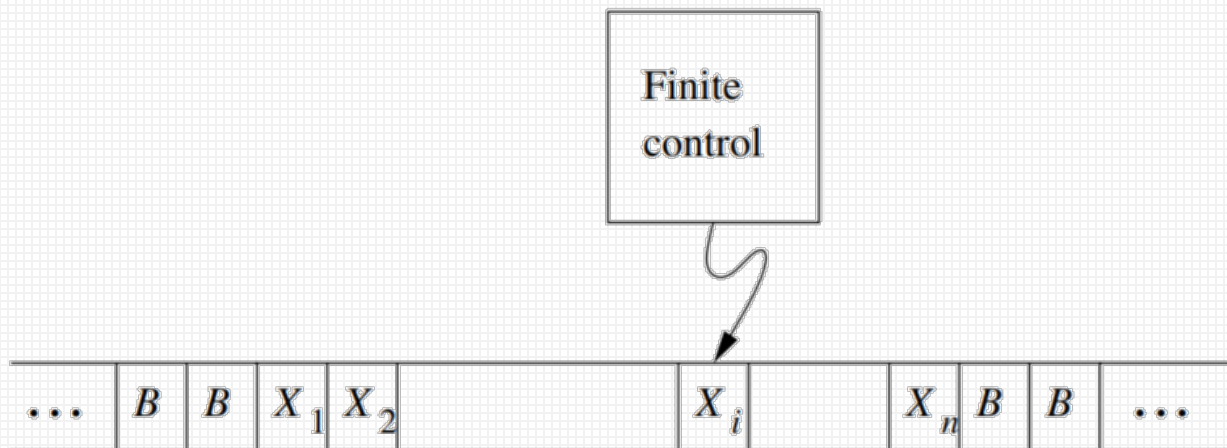
$L_1 = \{a^m b^n c^k \mid m, n, k \geq 1\}$ —————→ 正则语言（有穷自动机，正则表达式）

$L_2 = \{a^m b^m c^k \mid m, k \geq 1\}$

$L_3 = \{a^m b^m c^m \mid m \geq 1\}$

—————→ 上下文无关语言（上下文无关文法，下推自动机）

—————→ 图灵机所能识别的语言（图灵机）



- 图灵机有一个有穷控制器，一条两端无穷的输入输出带和一个带头(tape head)。
- 带划分为单元格，每个单元格可以放置一个符号。
 - 初始时，带上的符号由输入符号(input symbol)和空格符号B(blank symbol)组成
 - 空格符号是不同于输入符号的带符号(tape symbol)。除了空格符号外，也可以定义其他的带符号(tape symbol)
- 带头每次根据当前状态和带头处单元格的符号内容，根据转移规则选择下一个动作。
 - 每个动作都包括下一个状态，修改带头处单元格的符号以及带头向左或向右移动一个单元格。
 - 带头最开始指向最左边的非空符号，即图中的 X_1

- 图灵机(TM, Turing Machine) M 为七元组

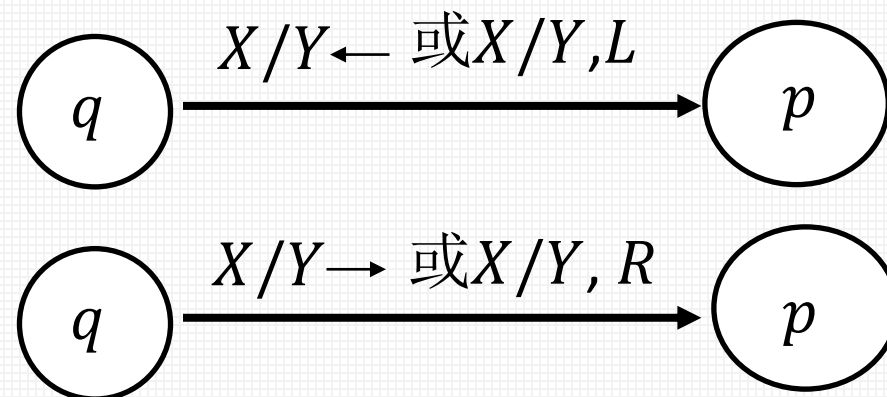
$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

- (1) Q : 有穷状态集
- (2) Σ : 有穷输入符号集(input alphabet)
- (3) Γ : 有穷带符号集(tape alphabet), 且总有 $\Sigma \subseteq \Gamma$
- (4) $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$: 转移函数
- (5) $q_0 \in Q$: 初始状态
- (6) $B \in \Gamma - \Sigma$: 空格/空白符号(Blank Symbol)
- (7) $F \subseteq Q$: 接受状态集

图灵机的动作及状态转移图

- 有穷控制器状态为 q 且带头读入符号 X 时，若有
$$\delta(q, X) = (p, Y, L/R)$$

则状态移到 p ,带头写符号 Y 并左或右移一格。图示为



- 和PDA类似，可以定义图灵机的瞬时描述ID，用于刻画图灵机的动作。
- 定义:用全部非空格符号、当前状态及带头位置, 定义图灵机的瞬时描述(ID):

$$X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n$$

- (1) q 是图灵机的当前状态;
 - (2) 带头在左起第 i 个非空格符 X_i 上;
 - (3) $X_1 X_2 \cdots X_n$ 是从最左到最右非空格内容
- ID 转移符号 $\vdash_M$ 。

若某ID是从另一个经有限步转移而得到的, 记为 $\vdash_M^*$ 。若 M 已知, 简记为 $\vdash$ 和 $\vdash^*$ 。

- 左移动: 如果 $\delta(q, X_i) = (p, Y, L)$, 定义ID转移为

$$X_1 \cdots X_{i-1} \textcolor{blue}{q} X_i \cdots X_n \vdash_M X_1 \cdots X_{i-2} \textcolor{red}{p} X_{i-1} \textcolor{violet}{Y} X_{i+1} \cdots X_n$$

表示 M 做一次向左移动 (消除 X_i 打印 Y , 并且左移, 状态变成 p)

- 特殊情况

为表明带头指向了空格符 B ,
包含空格符 B 在ID中。

$$- i = 1, \textcolor{blue}{q} X_1 \cdots X_n \vdash_M \textcolor{red}{p} B \textcolor{violet}{Y} X_2 \cdots X_n$$

$$- i = n \text{ 且 } Y = B, X_1 \cdots X_{n-1} \textcolor{blue}{q} X_n \vdash_M X_1 \cdots X_{n-2} \textcolor{red}{p} X_{n-1}$$

原先的最右符号 X_n 被替换为了空格符 B , 成为右边无穷空格符的一部分, 因此从ID中去除。

- 右移动: 如果 $\delta(q, X_i) = (p, Y, R)$, 定义ID转移为

$$X_1 \cdots X_{i-1} q X_i \cdots X_n \vdash_M X_1 \cdots X_{i-1} Y p X_{i+1} \cdots X_n$$

表示 M 做一次向右移动 (消除 X_i 打印 Y , 并且右移, 状态变成 p)

- 特殊情况

为表明带头指向了空格符 B ,
包含空格符 B 在ID中。

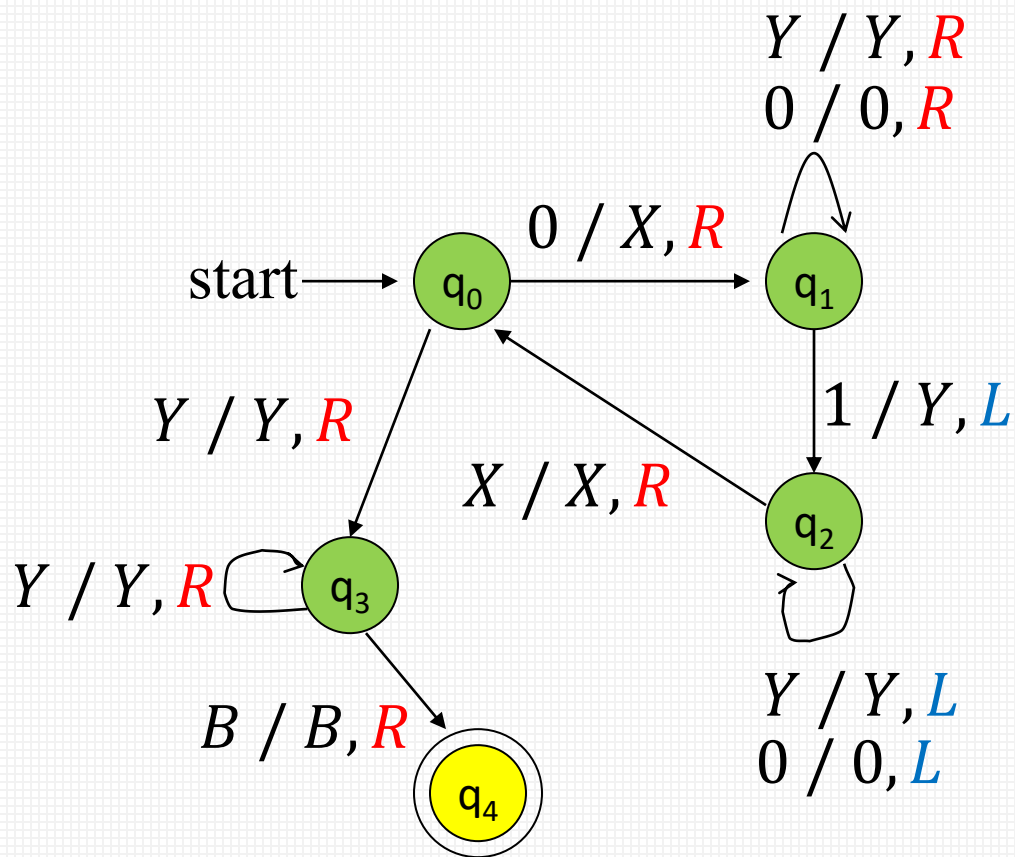
$$- i = n, X_1 \cdots X_{n-1} q X_n \vdash_M X_1 X_2 \cdots X_{n-1} Y p B$$

$$- i = 1 \text{ 且 } Y = B, q X_1 \cdots X_{n-1} X_n \vdash_M p X_2 \cdots X_n$$

原先的最左符号 X_1 被替换为了空格符 B , 成为左边无穷空格符的一部分, 因此从ID中去除。

示例

识别 $\{0^n 1^n | n \geq 1\}$ 的图灵机



1. 处于 q_0 , 若待读入的为0, 则将0标记为X, 向右移动
2. 一直向右移动(跳过0和Y), 直到遇到第一个1, 标记为Y, 改为向左移动
3. 一直向左移动(跳过Y和0), 直到遇到X, 改为向右移动
4. 如果下一个要读入的符号是0, 则跳转到第一步; 否则, 向右移动检查是否还有剩余的1。如果后面已经全是Y, 那么在空格符停下, 进入接受状态。

- 整数计算器：图灵机可以作为语言的识别器，也可以用作整数函数计算器。
- 例如，将整数 $i \geq 0$ 表示为字符串 0^i
- 若计算 k 个自变量 $i_1, i_2, \dots, i_k$ 的函数 f ，用
$$0^{i_1} 1 0^{i_2} \dots 1 0^{i_k}$$

作为图灵机 M 的输入

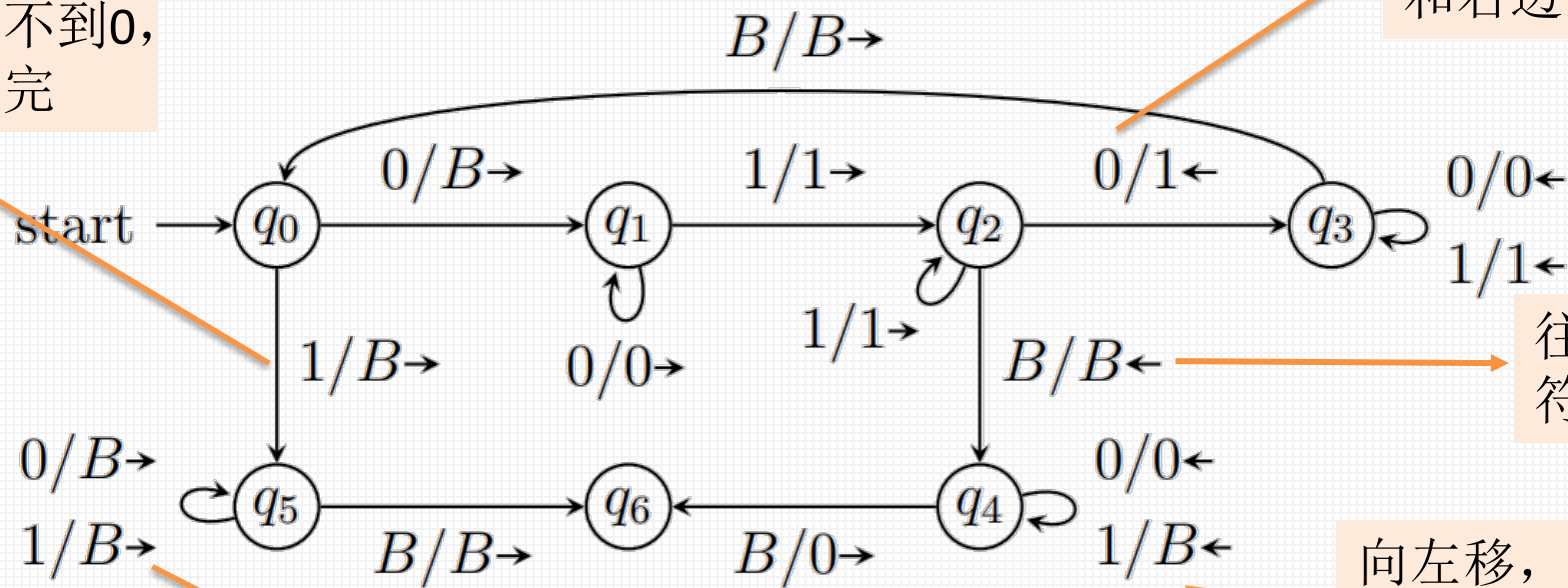
- 当 M 停机且输入带上为 0^m ，表示计算
$$f(i_1, i_2, \dots, i_k) = m$$

- 例2：设计整数真减法(monus or proper subtraction)的图灵机

$$m \dot{-} n = \begin{cases} m - n, & m \geq n \\ 0, & m < n \end{cases}$$

往左找不到0，
已标记完

正常的标记标记左边的0为空格符
和右边匹配的0为1.



- 定义：如果 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ 是一个图灵机, 则 M 接受的语言为

$$L(M) = \{w \mid w \in \Sigma^*, q_0 w \vdash^* \alpha p \beta, p \in F, \alpha, \beta \in \Gamma^*\}$$

- 初始时，输入串放在输入带上， M 处于 q_0 ，带头位于输入串的第一个字符上。 M 最终会进入某个终止（接受）状态。
 - 图灵机所接受的语言称为递归可枚举 (Recursively Enumerable) 语言。
- 一般总是假定当输入串 w 被接受时，即 $w \in L(M)$ 时， M 总会停机 (halt)，即没有下一个动作的定义。
- 而当 $w \notin L(M)$ ，可能有两种情况：
 - (1) 永远不停机，
 - (2) 停机但是不处于接受状态。

- 扩展图灵机
 - 状态中存储
 - 多道(multi-track)技术
 - 多带(multi-tape)技术
 - 非确定图灵机
- 受限图灵机
 - 半无穷带图灵机
 - 多栈机器

习题练习

习题练习-确定有限自动机的设计

方法一、（1）从初始状态开始，对每个状态，依次确定每个字符作为输入时，可能到达的状态：

（a）如果到达的状态是之前已出现的状态，建立当前状态到应到达状态的边；

（b）如果到达的状态是之前未出现的状态，增加新的状态，建立当前状态到新增状态的边。

（2）确定终止状态

方法二：从初始状态开始，重点关注可以到达结束状态的路径，然后对每个节点补充其他字符作为输入时的边和状态

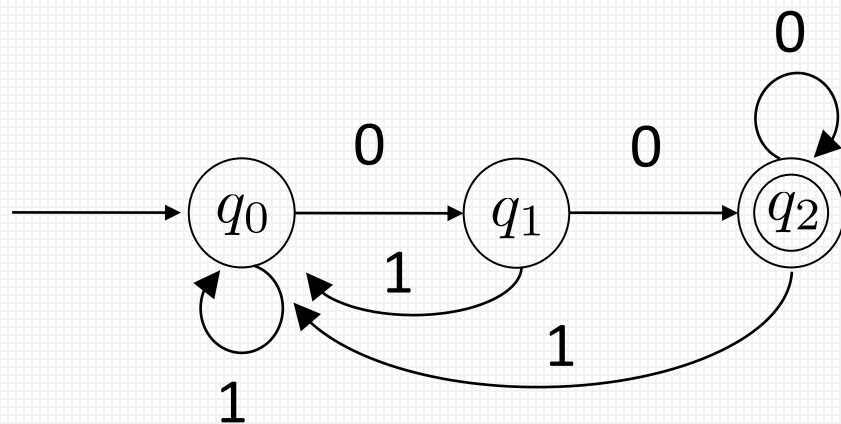
例1：设计定义在字母表 $\{0, 1\}$ 上的所有以00结尾的串的集合的DFA

正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

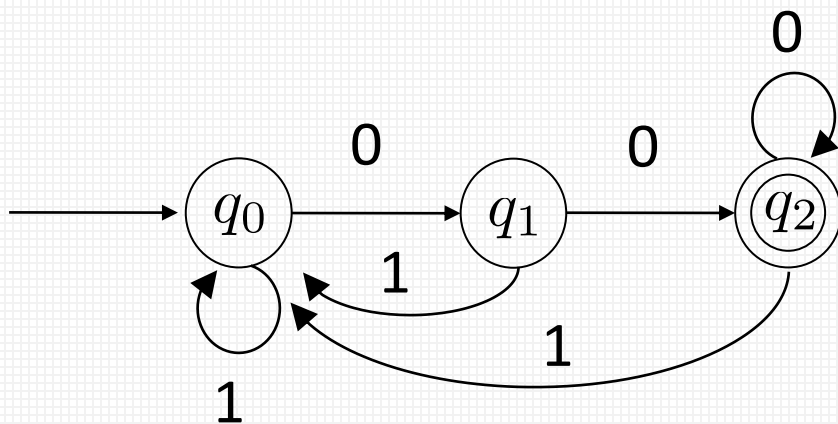
例1：设计定义在字母表 $\{0, 1\}$ 上的所有以00结尾的串的集合的DFA。



解：首先画出三个状态，接受两个连续的0到达终止状态。
由于DFA要求每个节点的出边必须包含字母表上的所有符号，因此，必须要补上所缺的标为1出边，同时也不能忘记终止状态还有一个标为0的出边。最后补上状态编号。

确定有限自动机

例1：设计定义在字母表 $\{0, 1\}$ 上的所有以00结尾的串的集合的DFA。



解：三个状态其实都有明确的含义。 q_0 表示出现第一对连续的0之前的任意字符串。 q_1 表示第一对连续的0之中的第一个0，一旦在该状态下碰到了1，必须转回到 q_0 。 q_2 则是终止状态，可以接受一对连续的0，也可以接受更多的连续的0。

例2：设计DFA接受以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中至少包含一个子串 } 010\}$$

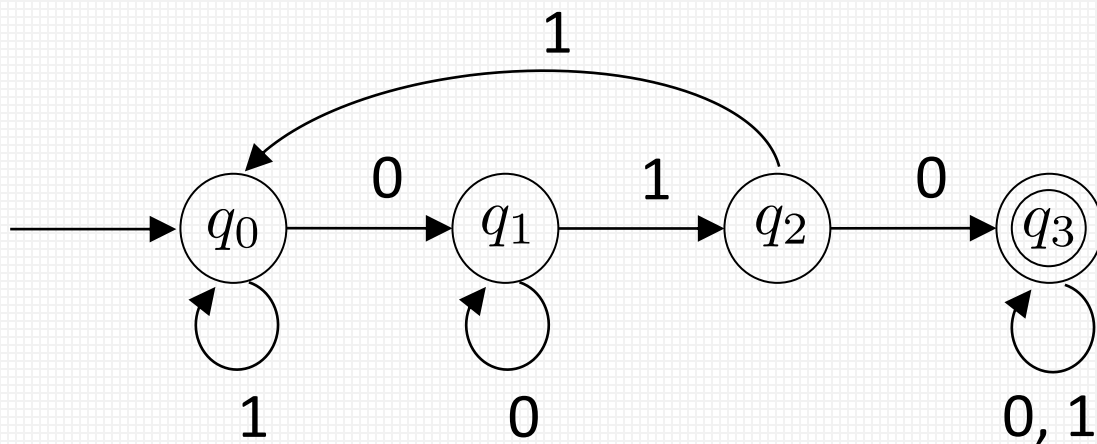
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例2：设计DFA接受以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中至少包含一个子串 } 010\}$$



解：这道题可以采用类似与例1的技巧，首先确定主线上的节点和边，然后再补所缺的出边。一定要想清楚出边应该指向哪个节点。做完后可以列举几个符合条件的串检查一下。

例3：设计DFA接受以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中不包含子串 } 010\}$$

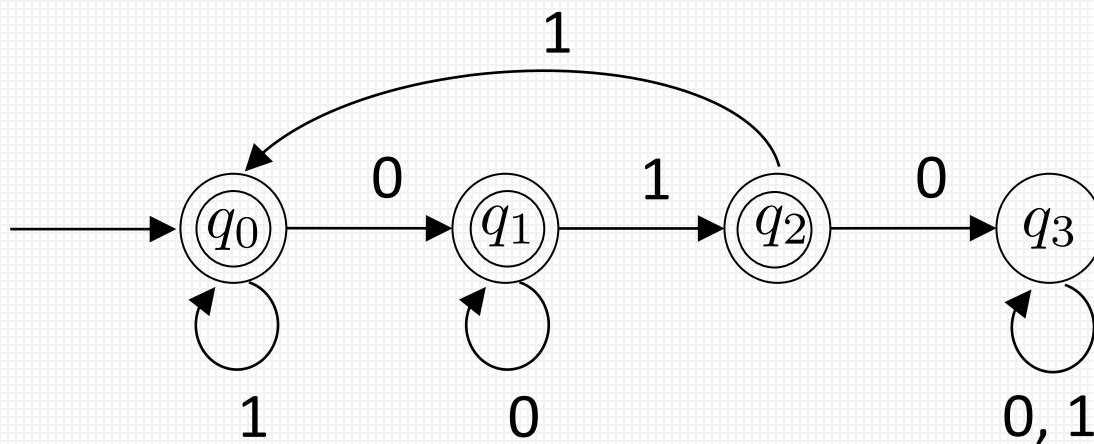
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例3：设计DFA接受以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中不包含子串 } 010\}$$



解：这道题看上去似乎无从下手，怎么表示“不包含”？其实可以从补集的角度去考虑，体现在DFA上，就是保留例2中DFA的结构，只不过互换终结状态和非终结状态。

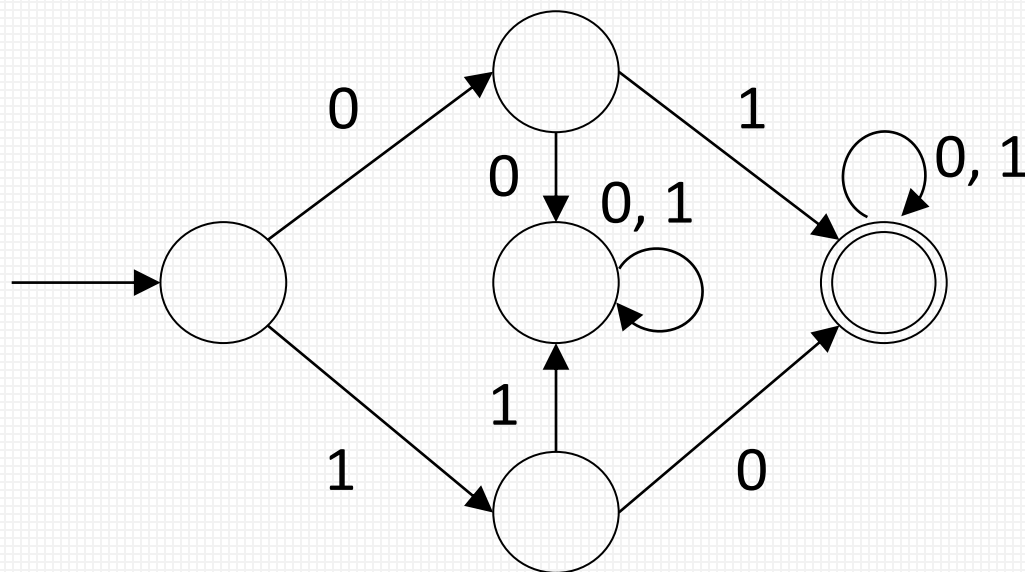
例4：设计DFA表示所有长度至少为2且头两个字符不同的0和1组成的串的集合。

正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例4：设计DFA表示所有长度至少为2且头两个字符不同的0和1组成的串的集合。



解：思路和前面一样，首先完成主线的构建，然后再对每个节点补出边。这里专门引入了一个节点充当“垃圾箱”，收无效的字符串，这是因为DFA要求每个节点必须有出边。

例5：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge \text{子串}01\text{在}x\text{中出现的次数为奇数}\}$$

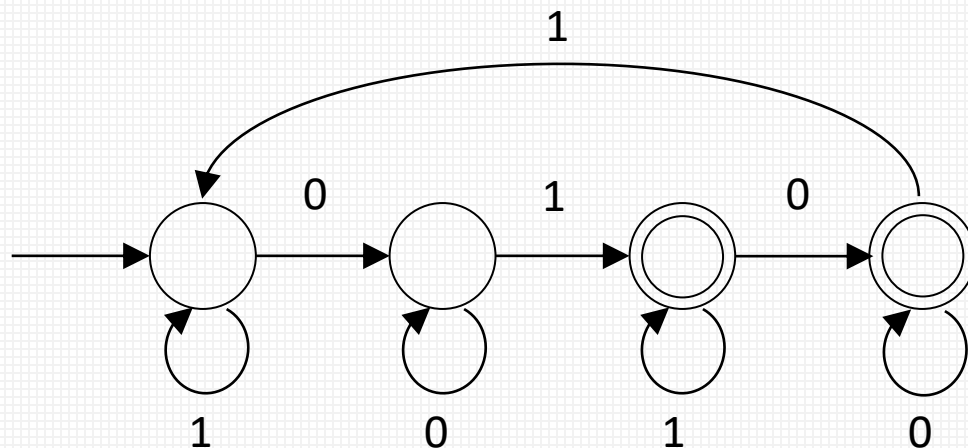
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例5：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge \text{子串}01\text{在}x\text{中出现的次数为奇数}\}$$



解：此题的难点在于如何实现奇数。基本的策略是即将形成偶数个01子串时就远离终结状态。需要注意的是，该串并不子串01的幂，只是包含子串01，所以还有其他子串。初始节点的含义是包含了偶数个子串01。

例6：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge 0 \text{ 的个数与 } 1 \text{ 的个数之和为奇数} \}$$

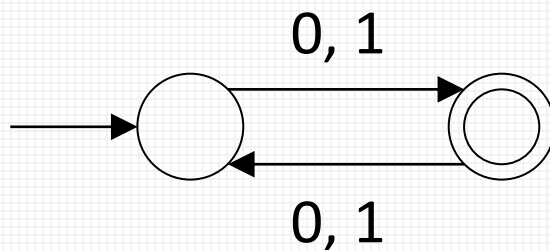
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例6：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge 0 \text{ 的个数与 } 1 \text{ 的个数之和为奇数}\}$$



解：这道题比较简单。题目等价于字符串的长度是奇数，这样就类似于电灯开关，比如按奇数次才处于开的状态。因此，很容易设计相应的DFA。

例7：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge 0 \text{的个数是偶数, } x \text{的长度也是偶数}\}$$

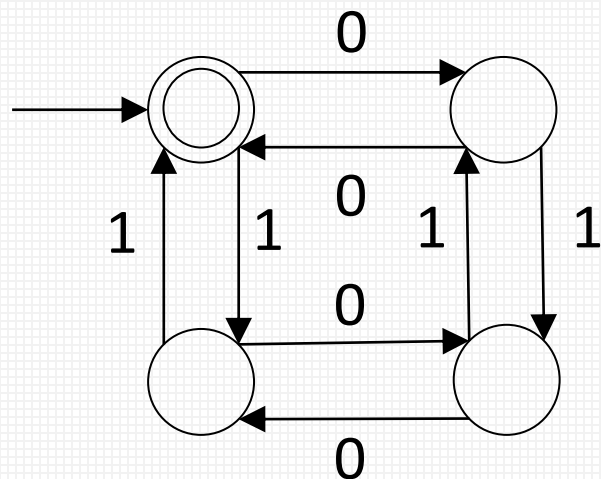
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例7：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge 0 \text{ 的个数是偶数, } x \text{ 的长度也是偶数}\}$$



解：题目等价于0的个数是偶数，1的个数也是偶数。根据0和1的奇、偶性，可以划分为4种状态相互关联。这样就比较容易设计DFA了。

例8：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中包含相同个数的0和1} \wedge x \text{ 的每个前缀0和1的个数之差不超过1}\}$$

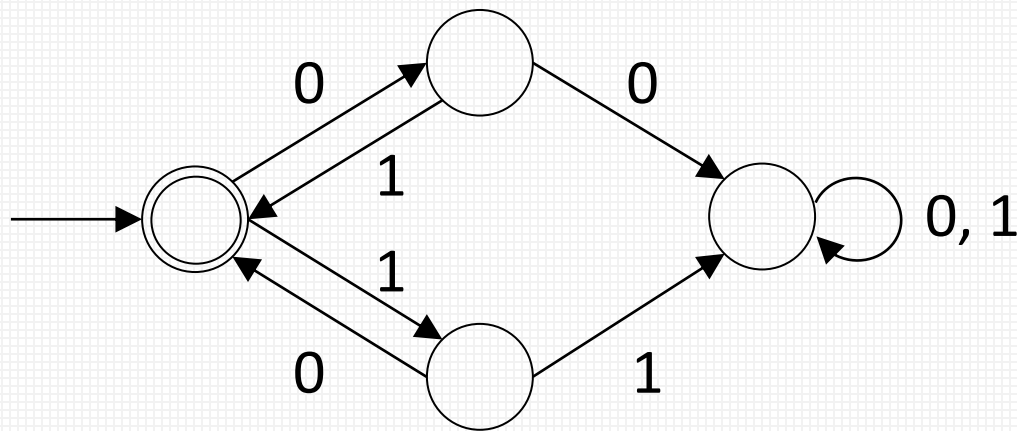
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例8：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 中包含相同个数的0和1} \wedge x \text{ 的每个前缀0和1的个数之差不超过1}\}$$



解：这道题的难点是如何实现每个前缀中0和1的个数之差不超过1。实现起来，就是只要出现0，紧接着要出现1，反之亦然。再加上垃圾回收节点，整个DFA就容易构建了。

例9：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 包含子串 } 01 \text{ 但不包含子串 } 11\}$$

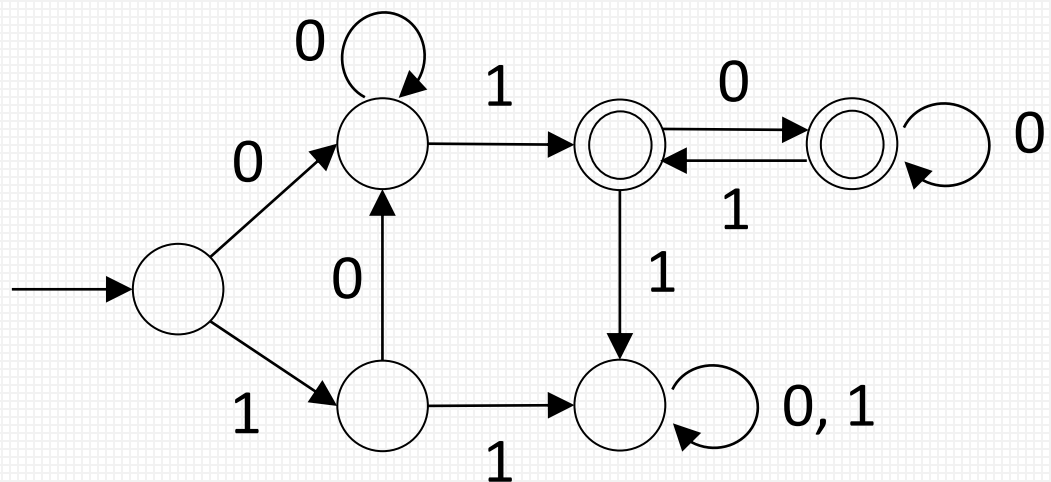
正常使用主观题需2.0以上版本雨课堂

作答

确定有限自动机

例9：设计DFA表示以下语言

$$L = \{x | x \in \{0, 1\}^* \wedge x \text{ 包含子串 } 01 \text{ 但不包含子串 } 11\}$$



解：这道题看上去很复杂，既有包含关系，又有不包含关系。但基本策略不变，还是先把主线上的节点和连线确定，再补齐所有的出边。

习题练习-非确定有限自动机的设计

方法：从初始状态开始，重点关注可以到达结束状态的路径。

例1：设计NFA接受以下语言

$$L = \{w | w \in \{0, 1\}^* \wedge |w| \geq 3 \wedge w \text{ 中第2位和第3位不同} \}$$

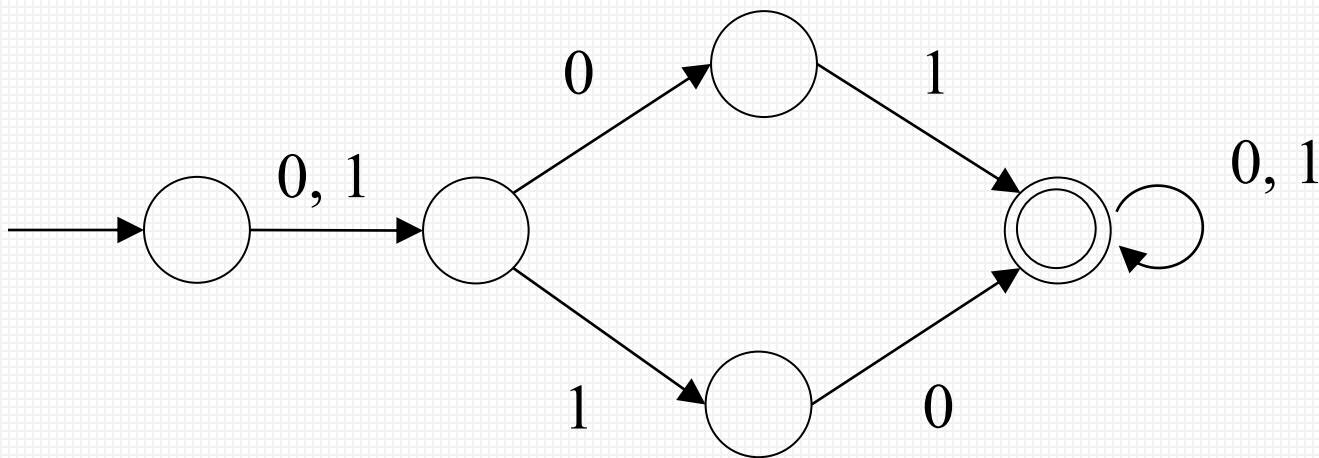
正常使用主观题需2.0以上版本雨课堂

作答

非确定有限自动机

例1：设计NFA接受以下语言

$$L = \{w | w \in \{0, 1\}^* \wedge |w| \geq 3 \wedge w \text{ 中第2位和第3位不同} \}$$



解：基本设计方法不变，还是沿着主线设计。第一个位置可是0或者1，第2位和第3位分成两条路线，确保题目要求的不同，最后是任意字符串。可见NFA的设计比DFA简单很多。

例2：考虑下面的

	ϵ	a	b	c
$\rightarrow p$	$\emptyset$	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
$*r$	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$

1. 计算每个状态的 ϵ 闭包。
2. 把这个自动机转换为DFA。

正常使用主观题需2.0以上版本雨课堂

作答

非确定有限自动机

例2：考虑下面的

	ϵ	a	b	c
$\rightarrow p$	$\emptyset$	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
$*r$	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$

1. 计算每个状态的 ϵ 闭包。
2. 把这个自动机转换为DFA。

解：

	a	b	c
$\rightarrow \{p\}$	$\{p\}$	$\{p, q\}$	$\{p, q, r\}$
$\{p, q\}$	$\{p, q\}$	$\{p, q, r\}$	$\{p, q, r\}$
$*\{p, q, r\}$	$\{p, q, r\}$	$\{p, q, r\}$	$\{p, q, r\}$

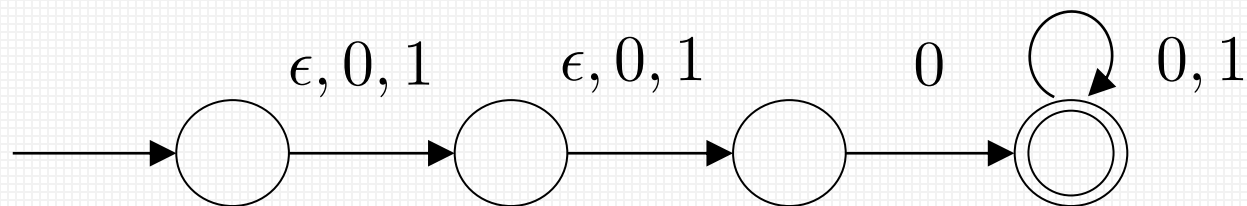
例 3：设计 ϵ -NFA 接受所有由0和1组成的串，要求长度至少为1，且前三个符号里面至少有一个0。

正常使用主观题需2.0以上版本雨课堂

作答

非确定有限自动机

例 3：设计 ϵ -NFA 接受所有由0和1组成的串，要求长度至少为1，且前三个符号里面至少有一个0。



解： ϵ -NFA 特别适合处理这种情况。

允许前两个符号出现空符号，自然就实现了题目的要求。

如果用DFA或者NFA的话，设计起来难度会更大。

例 4：设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 1 \wedge w \text{ 后3位中至少有1位不是 } c\}$$

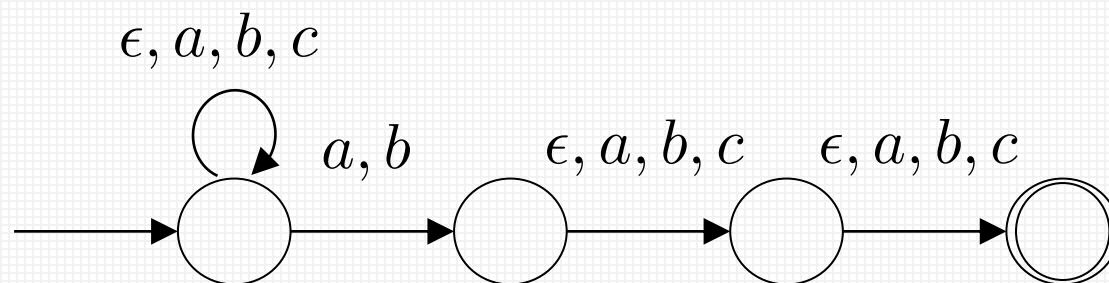
正常使用主观题需2.0以上版本雨课堂

作答

非确定有限自动机

例 4：设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 1 \wedge w \text{ 后3位中至少有1位不是 } c\}$$



解：题目等价于“后3位至少有1位是a或b”，这样就把看似很难处理的“不是c”转化为我们非常熟悉的形式，很容易给出相应的 ϵ -NFA。

例 5：设计 ϵ -NFA 接受以下语言

$$L = \{w \mid w \in \{a, b, c\}^* \wedge |w| \geq 2 \wedge w \text{ 从第2位至第4位至少有一位不是 } c\}$$

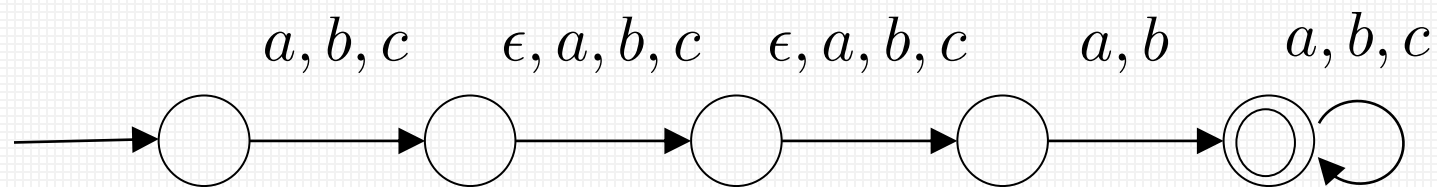
正常使用主观题需2.0以上版本雨课堂

作答

非确定有限自动机

例 5: 设计 ϵ -NFA 接受以下语言

$$L = \{w \mid w \in \{a, b, c\}^* \wedge |w| \geq 2 \wedge w \text{ 从第2位至第4位至少有一位不是 } c\}$$



例 6: 设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$$

w 的前 5 位至少有一个子串 $bac \wedge$

w 的后 5 位至少有一个子串 $acb\}$

正常使用主观题需 2.0 以上版本雨课堂

作答

非确定有限自动机

例 6: 设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$$

w 的前5位至少有一个子串 bac $\wedge$

w 的后5位至少有一个子串 acb $\}$

解: 这道题的难点在于需要充分考虑所有的情况。第1种情况是 bac 出现在 acb 的前面并且没有重叠。如

$$\underbrace{\dots bac \dots}_{\text{前5位}} \dots \overbrace{\dots acb \dots}^{\text{后5位}}$$

这是最容易想到的情况, 但还有更多的情况需要考虑。

非确定有限自动机

例 6: 设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$$

w 的前5位至少有一个子串 $bac \wedge$

w 的后5位至少有一个子串 $acb\}$

解: 由于两个子串有公共子串 ac , 因此第2种情况是 bac 在 acb 的前面并且两者交叠。



不仅如此, 更有第3种情况, 也是最容易漏掉的情况。

非确定有限自动机

例 6: 设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$$

w 的前5位至少有一个子串 bac $\wedge$

w 的后5位至少有一个子串 acb $\}$

解: 第3种情况是 bac 在 acb 的后面, 两者有公共子串 b , 整个串的长度为5。



后5位

acbabc

前5位

这三种情况是或的关系, 需要体现在 ϵ -NFA 中。

非确定有限自动机

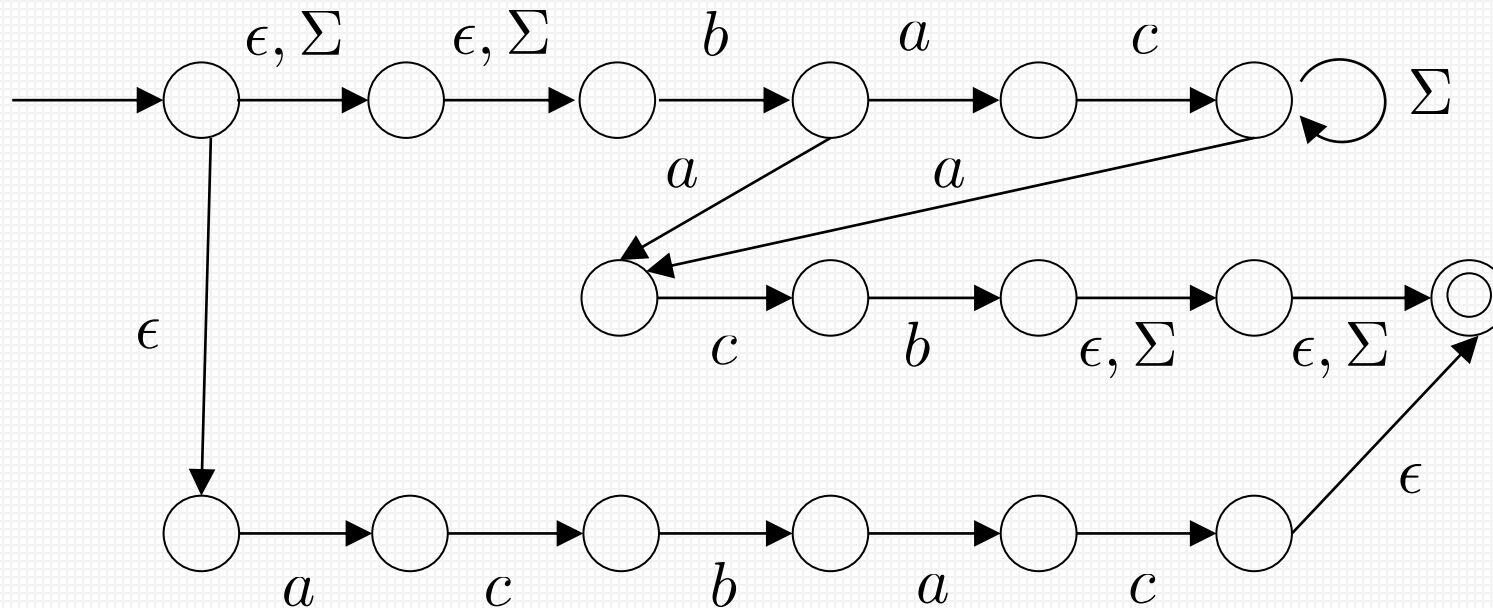
例 6: 设计 ϵ -NFA 接受以下语言

$$L = \{w | w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$$

w 的前 5 位至少有一个子串 bac $\wedge$

w 的后 5 位至少有一个子串 acb $\}$

解:



习题练习-正则表达式的设计

例1：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 长度为2且第2个字符是1}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例1: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 长度为2且第2个字符是1}\}$$

该语言的字母表包含0和1。

解: 可以分成两种情况。第一种情况是首字符是0:

01

第二种情况是首字符是1:

11

合并两种情况可以得到:

01 + 11

根据分配律:

$(0 + 1)1$

例2：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 至少包含3个连续的0} \}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例2：设计正则表达式 E 使得

$$L(E) = \{w | w \text{至少包含3个连续的0}\}$$

该语言的字母表包含0和1。

解：这道题要求至少包含3个连续的0，所以可以先写出来

000

由于左右可以是任意字符串，即任意长度、任意字符，因此可以利用星闭包来表示左右两侧。最终结果是：

$$(0 + 1)^*000(0 + 1)^*$$

例3：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 以 } 01 \text{ 结尾}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例3：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 以 } 01 \text{ 结尾}\}$$

该语言的字母表包含0和1。

解：先给出结尾的01，再补上左侧的任意字符串，因此结果为：

$$E = (0 + 1)^* 01$$

例4：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 以1开头且以0结尾} \}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例4：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 以1开头且以0结尾} \}$$

该语言的字母表包含0和1。

解：除了开头和结尾以外的内部为任意字符串。

$$E = 1(0 + 1)^*0$$

例5：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的开头字符和结尾字符相同} \}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例5：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的开头字符和结尾字符相同} \}$$

该语言的字母表包含0和1。

解：区分三种情况，

第一种情况是开头字符和结尾字符都是0：

$$0(0 + 1)^*0$$

第二种情况是开头字符和结尾字符都是1：

$$1(0 + 1)^*1$$

第三种情况是字符串的长度为1。因此结果为：

$$0(0 + 1)^*0 + 1(0 + 1)^*1 + 0 + 1$$

例6：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 由交替的0和1组成}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例6：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 由交替的0和1组成}\}$$

该语言的字母表包含0和1。

解：这道题一个直观的想法是 $E = (\mathbf{01})^*$ ，但是这样仅包含了以0开头、以1结尾的情况。需要考虑四种情况：

(1) 以0开头、以1结尾： $E_1 = (\mathbf{01})^*$

(2) 以1开头、以0结尾： $E_2 = (\mathbf{10})^*$

(3) 以0开头、以0结尾： $E_3 = \mathbf{0(10)^*}$

(4) 以1开头、以1结尾： $E_4 = \mathbf{1(01)^*}$

正则表达式

例6: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 由交替的0和1组成}\}$$

该语言的字母表包含0和1。

解: 合并这四种情况可以得到

$$E = (\mathbf{01})^* + (\mathbf{10})^* + \mathbf{0(10)^*} + \mathbf{1(01)^*}$$

上面的式子是否可以写成更紧凑的形式呢? 可以利用分配律来实现。

正则表达式

例6: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 由交替的0和1组成}\}$$

该语言的字母表包含0和1。

解: 利用分配律来处理

$$\begin{aligned} & (\mathbf{01})^* + (\mathbf{10})^* + \mathbf{0}(\mathbf{10})^* + \mathbf{1}(\mathbf{01})^* \\ &= (\epsilon + \mathbf{1})(\mathbf{01})^* + (\mathbf{10})^* + \mathbf{0}(\mathbf{10})^* \\ &= (\epsilon + \mathbf{1})(\mathbf{01})^* + (\mathbf{10})^* + (\mathbf{01})^*\mathbf{0} \\ &= (\epsilon + \mathbf{1})(\mathbf{01})^*(\epsilon + \mathbf{0}) + (\mathbf{10})^* \\ &= (\epsilon + \mathbf{1})(\mathbf{01})^*(\epsilon + \mathbf{0}) + \epsilon + \mathbf{1}(\mathbf{01})^*\mathbf{0} \\ &= (\epsilon + \mathbf{1})(\mathbf{01})^*(\epsilon + \mathbf{0}) \end{aligned}$$

正则表达式

例6: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 由交替的0和1组成}\}$$

该语言的字母表包含0和1。

解: 还有另一种处理方式

$$\begin{aligned} & (\mathbf{01})^* + (\mathbf{10})^* + \mathbf{0(10)^*} + \mathbf{1(01)^*} \\ &= (\epsilon + \mathbf{0})(\mathbf{10})^* + (\mathbf{01})^* + \mathbf{1(01)^*} \\ &= (\epsilon + \mathbf{0})(\mathbf{10})^* + (\mathbf{01})^* + (\mathbf{10})^*\mathbf{1} \\ &= (\epsilon + \mathbf{0})(\mathbf{10})^*(\epsilon + \mathbf{1}) + (\mathbf{01})^* \\ &= (\epsilon + \mathbf{0})(\mathbf{10})^*(\epsilon + \mathbf{1}) + \epsilon + \mathbf{0(10)^*1} \\ &= (\epsilon + \mathbf{0})(\mathbf{10})^*(\epsilon + \mathbf{1}) \end{aligned}$$

例7：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 从右端数第5个位置是1}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例7：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 从右端数第5个位置是1}\}$$

该语言的字母表包含0和1。

解：倒数第5个位置是1，那么倒数前4个位置必然或者为0，或者为1。前面的字符串则可以是任意字符串。因此：

$$E = (0 + 1)^* 1 (0 + 1)^4$$

例8：设计正则表达式 E 使得

$$L(E) = \{w | w \text{的前5位至少包含一个} 1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例8：设计正则表达式 E 使得

$$L(E) = \{w | w \text{的前5位至少包含一个} 1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

解：这道题的难点在于如何是实现前5位至少包含一个字符。

注意：字符串的长度并不一定超过5。此处的技巧是巧妙地使用空串：

$$E = (\mathbf{0 + 1 + \epsilon})^4 \mathbf{1(0 + 1)^*}$$

例9：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的后5位至少包含一个 } 1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例9：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的后5位至少包含一个 } 1 \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

解：

$$E = (\mathbf{0} + \mathbf{1})^* \mathbf{1} (\mathbf{0} + \mathbf{1} + \epsilon)^4$$

例10：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 中至少包含一个 } a \text{ 和一个 } b\}$$

该语言的字母表包含 a 、 b 和 c 。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例10：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 中至少包含一个 } a \text{ 和一个 } b\}$$

该语言的字母表包含 a 、 b 和 c 。

解：由于 w 中至少包含一个 a 和 b ，不妨就考虑从左往右数的第一个 a 和第一个 b 。那么可以分为两种情况。

第一种情况是第一个 a 在第一个 b 之前。那么，在第一个 a 之前必然只可能出现 c ，在第一个 a 和第一个 b 之间必然只可能出现 a 和 c ，在第一个 b 之后可以出现任意字符串。因此，可以写成

$$\mathbf{c^* a(a + c)^* b(a + b + c)^*}$$

正则表达式

例10：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 中至少包含一个 } a \text{ 和一个 } b\}$$

该语言的字母表包含 a 、 b 和 c 。

解：第二种情况是第一个 b 在第一个 a 之前，可以类似处理

$$\mathbf{c^*b(b + c)^*a(a + b + c)^*}$$

因此，最终的结果是

$$E = \mathbf{c^*a(a + c)^*b(a + b + c)^* + c^*b(b + c)^*a(a + b + c)^*}$$

例11：设计正则表达式 E 使得

$L(E) = \{w | w \text{ 中每对相邻的0都出现在每对相邻的1之前} \}$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例11：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 中每对相邻的0都出现在每对相邻的1之前}\}$$

该语言的字母表包含0和1。

解：这道题看似无从下手。技巧是把字符串分成两部分，左部允许0相邻但不允许1相邻，右部允许1相邻但不允许0相邻，这样自然就实现了。

首先看允许0相邻但不允许1相邻，直接的写法是

$$(10 + 0)^*$$

但是没有考虑以1结尾的情况，因此可以写成

$$(10 + 0)^*(\epsilon + 1)$$

正则表达式

例11：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 中每对相邻的0都出现在每对相邻的1之前}\}$$

该语言的字母表包含0和1。

解：再来看右部，允许1相邻但不允许0相邻

$$(01 + 1)^*(\epsilon + 0)$$

把两部分拼接起来可得到：

$$E = (10 + 0)^*(\epsilon + 1)(01 + 1)^*(\epsilon + 0)$$

例12：设计正则表达式 E 使得

$$L(E) = \{w | w = xy \wedge x, y \in \{0, 1\}^* \wedge |y| = 3 \wedge y = y^R\}$$

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例12: 设计正则表达式 E 使得

$$L(E) = \{w | w = xy \wedge x, y \in \{0, 1\}^* \wedge |y| = 3 \wedge y = y^R\}$$

解: 一般而言正则表达式是没法处理回文的, 但是这道题中的回文长度受限, 因此是可以全枚举的:

$$E = (0 + 1)^*(000 + 010 + 111 + 101)$$

例13：设计正则表达式 E 使得

$$L(E) = \{w | w \text{中既不包含子串} 00, \text{也不包含子串} 11\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例13：设计正则表达式 E 使得

$$L(E) = \{w | w \text{中既不包含子串} 00, \text{也不包含子串} 11\}$$

该语言的字母表包含0和1。

解：这道题实际上和前面介绍的例题是等价的。要注意同一问题的不同表述。

$$E = (\epsilon + \mathbf{1})(\mathbf{01})^*(\epsilon + \mathbf{0})$$

例14：设计正则表达式 E 使得

$$L(E) = \{0^m 1^n \mid m \geq 0 \wedge n \geq 0 \wedge m + n \text{是偶数}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例14：设计正则表达式 E 使得

$$L(E) = \{0^m 1^n \mid m \geq 0 \wedge n \geq 0 \wedge m + n \text{ 是偶数} \}$$

该语言的字母表包含0和1。

解： $m + n$ 是偶数，只有两种情况。第一种情况是 m 和 n 都是偶数：

$$(00)^*(11)^*$$

第二种情况是 m 和 n 都是奇数：

$$(00)^*01(11)^*$$

合并两种情况可以得到：

$$(00)^*(01 + \epsilon)(11)^*$$

例15：设计正则表达式 E 使得

$$L(E) = \{w \mid \text{当 } w \text{ 以 } 0 \text{ 结尾时, 其长度是奇数} \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例15：设计正则表达式 E 使得

$$L(E) = \{w \mid \text{当 } w \text{ 以 } 0 \text{ 结尾时, 其长度是奇数} \wedge |w| \geq 1\}$$

该语言的字母表包含0和1。

解：需要考虑两种情况。第一种情况是以0结尾：

$$((0 + 1)^2)^*0$$

第二种情况是以1结尾：

$$(0 + 1)^*1$$

因此，最终结果是

$$((0 + 1)^2)^*0 + (0 + 1)^*1$$

例16: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{的前5位至少有一个子串} 00 \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

正则表达式

例16: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{的前5位至少有一个子串} 00 \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

解:

$$(0 + 1 + \epsilon)^3 00 (0 + 1)^*$$

例17：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的第2位至第5位至少有一个1} \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例17：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 的第2位至第5位至少有一个1} \wedge |w| \geq 2\}$$

该语言的字母表包含0和1。

解：对于并不是顺数或者倒数第几位的情况，需要加上位置的偏移：

$$E = (0 + 1)(0 + 1 + \epsilon)^3 1 (0 + 1)^*$$

例18：设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 至少包含3个1, 且倒数第3位为1}\}$$

该语言的字母表包含0和1。

正常使用主观题需2.0以上版本雨课堂

作答

正则表达式

例18: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{ 至少包含3个1, 且倒数第3位为1}\}$$

该语言的字母表包含0和1。

解: 像这样的题目, 要分情况讨论。这里需要考虑三种情况。

第一种情况是剩下两个1都在倒数第3位左侧:

$$(0 + 1)^* 1 (0 + 1)^* 1 (0 + 1)^* 1 0 0$$

第二种情况是剩下两个1分别在倒数第3位左侧和右侧:

$$(0 + 1)^* 1 (0 + 1)^* 1 (01 + 10)$$

第三种情况是剩下两个1都在倒数第3位右侧:

$$(0 + 1)^* 1 1 1$$

正则表达式

例18: 设计正则表达式 E 使得

$$L(E) = \{w | w \text{至少包含3个1, 且倒数第3位为1}\}$$

该语言的字母表包含0和1。

解: 因此, 最终的结果是

$$\begin{aligned} E = & (0 + 1)^* 1 (0 + 1)^* 1 (0 + 1)^* 1 0 0 + \\ & (0 + 1)^* 1 (0 + 1)^* 1 (0 1 + 1 0) + \\ & (0 + 1)^* 1 1 1 \end{aligned}$$

习题练习-上下文无关文法的设计

例1：请给出下列语言的一个上下文无关文法

$$L = \{a^m b^n \mid m \geq 0 \wedge n \geq 0 \wedge 3m \geq n \geq 2m\}$$

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法

例1：请给出下列语言的一个上下文无关文法

$$L = \{a^m b^n \mid m \geq 0 \wedge n \geq 0 \wedge 3m \geq n \geq 2m\}$$

解：这道题一眼看上去感觉无从下手，如何实现区间关系？

不管那么多，不妨先想办法实现 $n = 2m$

$$S \rightarrow aSbb$$

显然，这种递归形式可以保证2倍的关系。那么3倍的关系也很容易实现了

$$S \rightarrow aSbbb$$

那么如何实现区间关系呢？

上下文无关文法

例1：请给出下列语言的一个上下文无关文法

$$L = \{a^m b^n \mid m \geq 0 \wedge n \geq 0 \wedge 3m \geq n \geq 2m\}$$

解：幸运的是，当文法同时包含这两条规则是，恰好实现了区间关系

$$S \rightarrow aSbb$$

$$S \rightarrow aSbbb$$

这是因为两条规则的混用恰好能实现区间关系。加入基础部分，就得到了最终的文法

$$S \rightarrow aSbb \mid aSbbb \mid \epsilon$$

例2：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法

例2：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

解：在前面的例子里， a 和 b 的顺序是指定的，而这道题就难很多，因为对顺序没有要求，只要求数量相等。可以考虑三种情况。第1种情况是空串 $S \rightarrow \epsilon$ 。

第2种情况是字符串以 a 开头，则在剩下的字符串中 b 的数量必然比 a 多1个。对剩下的字符串重新计数，当第1次遇到 b 的数量比 a 多1个时停下来，可将剩下的字符串分成三部分：左部和右部中 a 的数量和 b 相等，中间是一个单独的分割点 b 。

上下文无关文法

例2：请给出下列语言的一个上下文无关文法

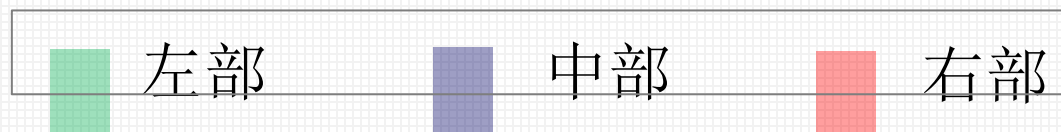
$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

解：不妨看几个具体的例子

aabbab

ababab

aaaabbbb



注意：中间部分作为分割点的 b 是唯一的。

因此，很容易用递归的形式来定义

$$S \rightarrow aSbS$$

上下文无关文法

例2：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) = \text{count}(w, b)\}$$

解：第3种情况是字符串以 b 开头，可以类似地递归定义

$$S \rightarrow bSaS$$

综上所述，该文法定义为

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

例3：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法

例3：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：由于使用了绝对值，因此要区分两种情况

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) - \text{count}(w, b) = 2\} \cup \\ \{w | w \in \{a, b\}^*, \text{count}(w, b) - \text{count}(w, a) = 2\}$$

首先考虑第一种情况，即 a 的数量比 b 多2个。依然可以采用之前的分割技术。从左往右扫描，计算 a 的数量和 b 的数量，当 a 的数量第1次比 b 的数量多1时，将该 a 确定为第一个分割点。

上下文无关文法

例3：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：然后将计数清零，类似地去找第2个分割点 a 。这两个分割点可以将字符串切成5个部分

*bb**aa**a**bb**aa**a**ba*

蓝色部分是切割点，红色部分两个元素的个数相等。

因此可以写成：

$$A \rightarrow CaCaC$$

$$C \rightarrow aCbC \mid bCaC \mid \epsilon$$

上下文无关文法

例3：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, |\text{count}(w, a) - \text{count}(w, b)| = 2\}$$

解：第二种情况是对称的，可以类似地处理。

因此，最终的文法是

$$S \rightarrow A \mid B$$

$$A \rightarrow CaCaC$$

$$B \rightarrow CbCbC$$

$$C \rightarrow aCbC \mid bCaC \mid \epsilon$$

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：可以用前面的策略，先把文法拆成两部分

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) > \text{count}(w, b)\} \cup \\ \{w | w \in \{a, b\}^*, \text{count}(w, a) < \text{count}(w, b)\}$$

首先考虑第一部分，即 a 的数量比 b 多。该怎么下手？

随机列几个符合条件的字符串，仔细观察

aab $aabaaba$ $baaab$ $abaaaaaba$

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：假设对于每个字符串从左往右扫描，边扫描边数 a 和 b 的数量，遇到第一次 a 的数量比 b 多1就停下来。这样可以把字符串分成三部分，左边的部分 a 的数量和 b 相等，中间部分只有1个 a ，右边的部分 a 的数量不低于 b

ab $ababab$ $baab$ $abaaab$

注意：中间部分作为分割点的 a 是唯一的。

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：如何实现 a 的数量不低于 b ？可以在数量相等的基础上增加 a 的数量。

$$D \rightarrow aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$

因此，第一种情况可以表示为

$$A \rightarrow CaD$$

$$C \rightarrow aCbC \mid bCaC \mid \epsilon$$

$$D \rightarrow aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：第二种情况是对称的，可以类似地处理。

因此，最终的文法是

$$S \rightarrow A \mid B$$

$$A \rightarrow CaD$$

$$B \rightarrow CbE$$

$$C \rightarrow aCbC \mid bCaC \mid \epsilon$$

$$D \rightarrow aDbD \mid bDaD \mid aD \mid Da \mid \epsilon$$

$$E \rightarrow aEbE \mid bEaE \mid bE \mid Eb \mid \epsilon$$

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w \mid w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：但是上面的答案比较冗余，尤其是后三个式子有些重复。
能不能写得更简单一些？以第一种情况为例，只要分割点两侧都满足 a 的数量不低于 b 即可。

abaaaba

这样可以简化一些

$$A \rightarrow CaC$$

$$C \rightarrow aCbC \mid bCaC \mid aC \mid Ca \mid \epsilon$$

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w | w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：按照这种方式得到的文法比之前少一个式子

$$S \rightarrow A \mid B$$

$$A \rightarrow CaC$$

$$B \rightarrow DbD$$

$$C \rightarrow aCbC \mid bCaC \mid aC \mid Ca \mid \epsilon$$

$$D \rightarrow aDbD \mid bDaD \mid bD \mid Db \mid \epsilon$$

上下文无关文法

例4：请给出下列语言的一个上下文无关文法

$$L = \{w \mid w \in \{a, b\}^*, \text{count}(w, a) \neq \text{count}(w, b)\}$$

解：不过这种方法的分割点不是唯一的。以下分割都合法。

*a*baaaaba

ab*a*aaaba

abaa*a*aba

abaaa*a*ba

abaaaa*a*ba

abaaaaab*a*

习题练习-下推自动机的设计

例1：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w \mid w \in \{a, b\}^* \text{ 且 } w \text{ 中任何前缀中 } a \text{ 的数量至少2倍于 } b \text{ 的数量}\}$$

正常使用主观题需2.0以上版本雨课堂

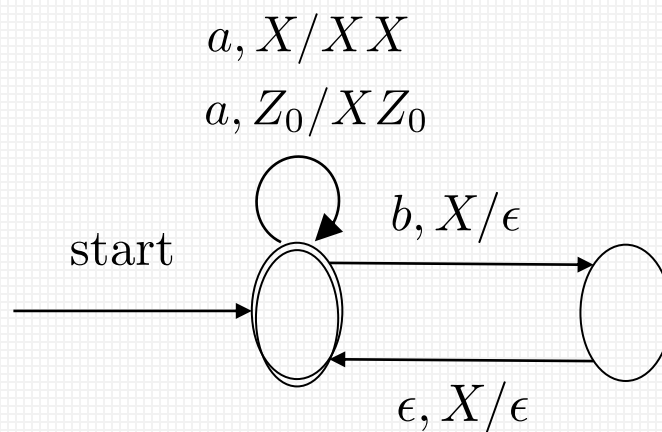
作答

下推自动机

例1：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w \mid w \in \{a, b\}^* \text{ 且 } w \text{ 中任何前缀中 } a \text{ 的数量至少 } 2 \text{ 倍于 } b \text{ 的数量}\}$$

解：可以考虑以终态接受的方式构造PDA。利用栈来进行数量上的控制，遇到符号 a 时向栈中压入元素，遇到符号 b 时从栈中弹出元素。为了实现“任何前缀中 a 的数量至少2倍于 b 的数量”，技巧是引入 ϵ 。

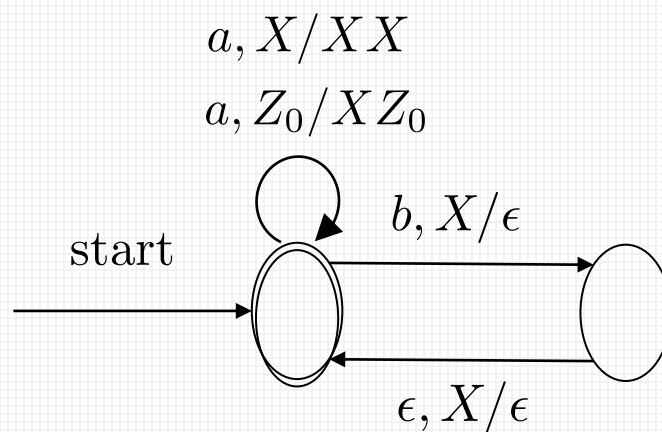


下推自动机

例1：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w \mid w \in \{a, b\}^* \text{ 且 } w \text{ 中任何前缀中 } a \text{ 的数量至少 } 2 \text{ 倍于 } b \text{ 的数量}\}$$

解：可以考虑以终态接受的方式构造PDA。利用栈来进行数量上的控制，遇到符号 a 时向栈中压入元素，遇到符号 b 时从栈中弹出元素。为了实现“任何前缀中 a 的数量至少2倍于 b 的数量”，技巧是引入 ϵ 。



例2：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w | w \in \{a, b\}^*, \text{ 且 } w \text{ 中 } a \text{ 的数量不等于 } b \text{ 的数量}\}$$

正常使用主观题需2.0以上版本雨课堂

作答

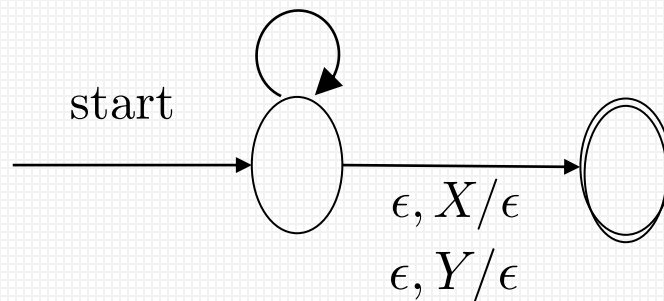
下推自动机

例2：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w | w \in \{a, b\}^*, \text{ 且 } w \text{ 中 } a \text{ 的数量不等于 } b \text{ 的数量}\}$$

解：还是构建终态接受的PDA。基本策略还是利用栈中元素来控制数量。我们可以用 X 来对 a 计数， Y 来对 b 计数。当 a 遇到 X 则压入一个 X ，当 a 遇到一个 Y 则弹出一个 Y ，当 b 遇到 Y 则压入一个 Y ，当 b 遇到 X 则弹出一个 X 。如果输入串处理完毕，栈中还有至少一个 X 或者 Y ，则表明 a 和 b 的数量一定不相等。

$$\begin{array}{ll} a, Z_0 / X Z_0 & b, Z_0 / Y Z_0 \\ a, X / X X & b, Y / Y Y \\ a, Y / \epsilon & b, X / \epsilon \end{array}$$



例3: 为文法 $S \rightarrow aAA, A \rightarrow aS \mid bS \mid a$ 构造等价的PDA。

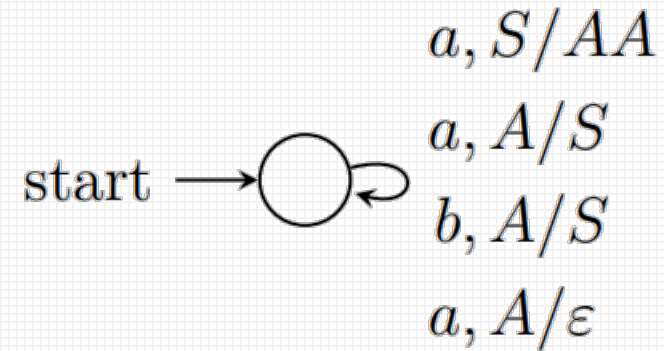
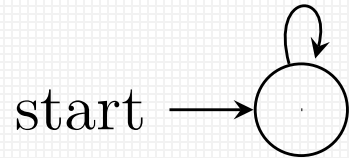
正常使用主观题需2.0以上版本雨课堂

作答

下推自动机

例3：为文法 $S \rightarrow aAA, A \rightarrow aS \mid bS \mid a$ 构造等价的PDA。

$\varepsilon, S/aAA$ $\varepsilon, A/aS$ $a, a/\varepsilon$
 $\varepsilon, A/a$ $\varepsilon, A/bS$ $b, b/\varepsilon$



注：GNF文法对应PDA的 δ 函数：
 $\delta(q, a, A) = \{(q, \beta) \mid A \rightarrow a\beta \in P\}$

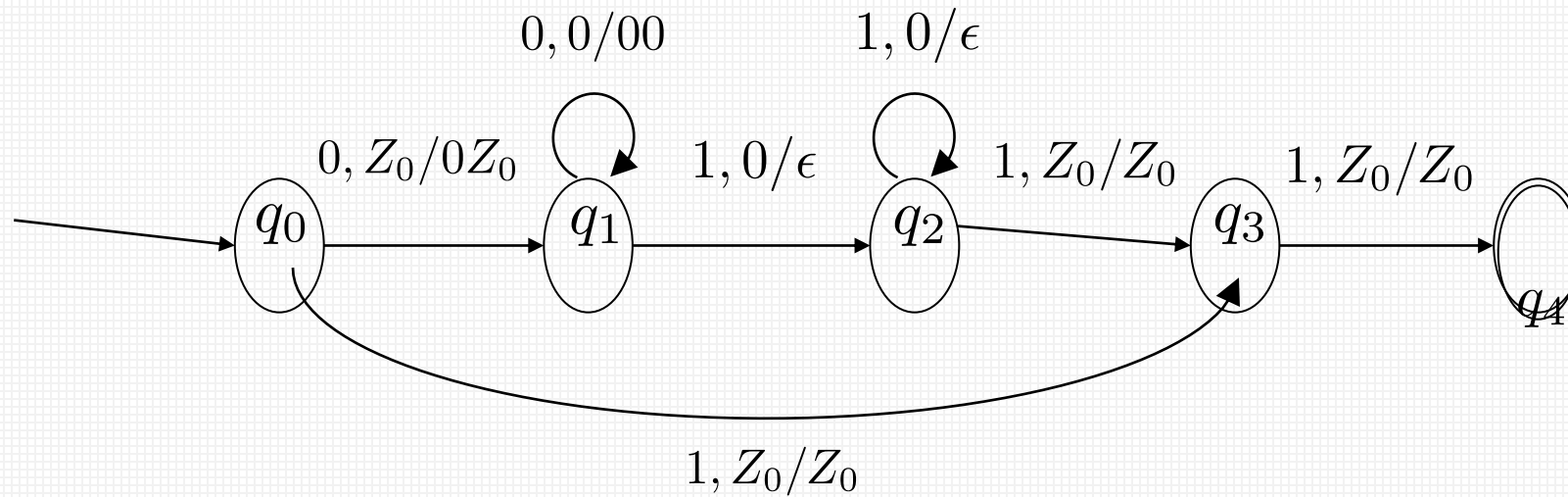
例4：构造一个DPDA，使其接受语言 $\{0^n 1^{n+2} \mid n \geq 0\}$ 。

正常使用主观题需2.0以上版本雨课堂

作答

下推自动机

例4：构造一个DPDA，使其接受语言 $\{0^n 1^{n+2} \mid n \geq 0\}$ 。



例5：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w | w \in \{a, b, c\}^*, \text{ 且 } w \text{ 中 } a \text{ 和 } b \text{ 的个数相等且不含连续的 } c\}$$

正常使用主观题需2.0以上版本雨课堂

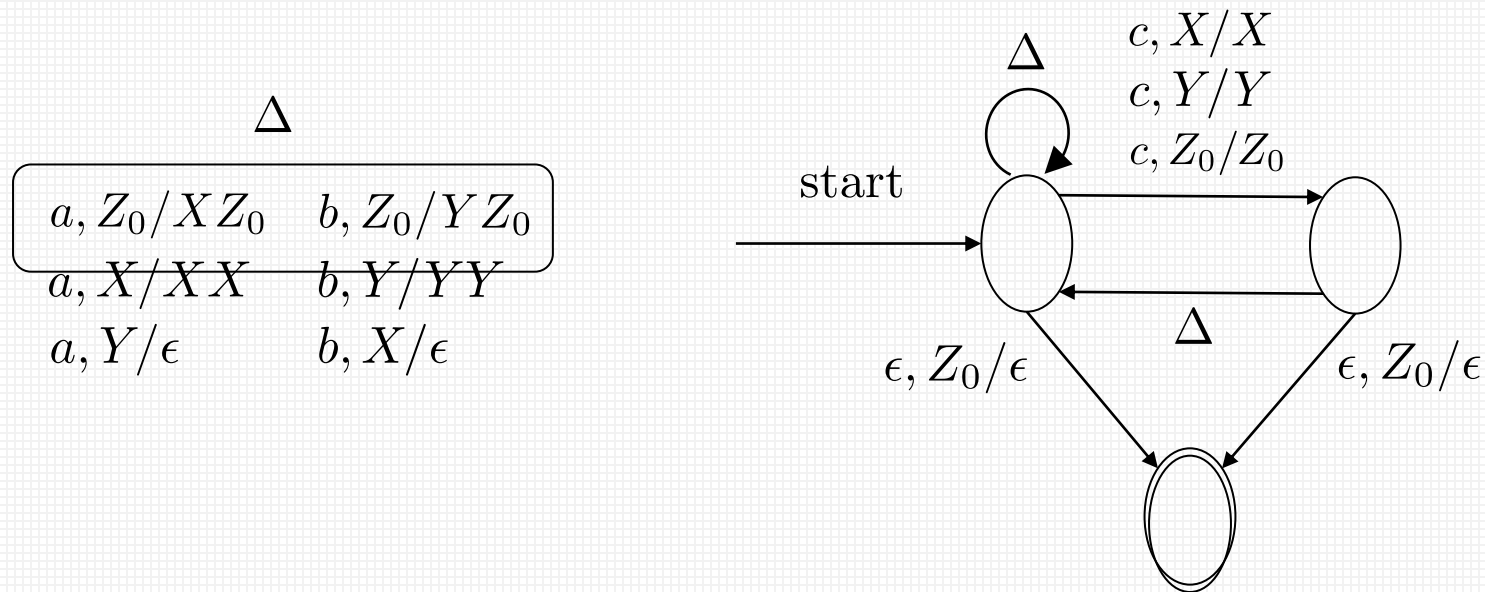
作答

下推自动机

例5：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{w | w \in \{a, b, c\}^*, \text{ 且 } w \text{ 中 } a \text{ 和 } b \text{ 的个数相等且不含连续的 } c\}$$

解：构造终态接受的PDA。可以借用例3的技巧，如果字符串处理完毕时栈中既没有 X 也没有 Y ，那么就实现了 a 和 b 的数量相等。为了不出现连续的 c ，只需每次出现 c 都马上出现 a 或 b 即可。



例6：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{a^n b^m c^k \mid n \geq 0, m \geq 0, k \geq 0 \text{ 且 } n + 2m = k\}$$

正常使用主观题需2.0以上版本雨课堂

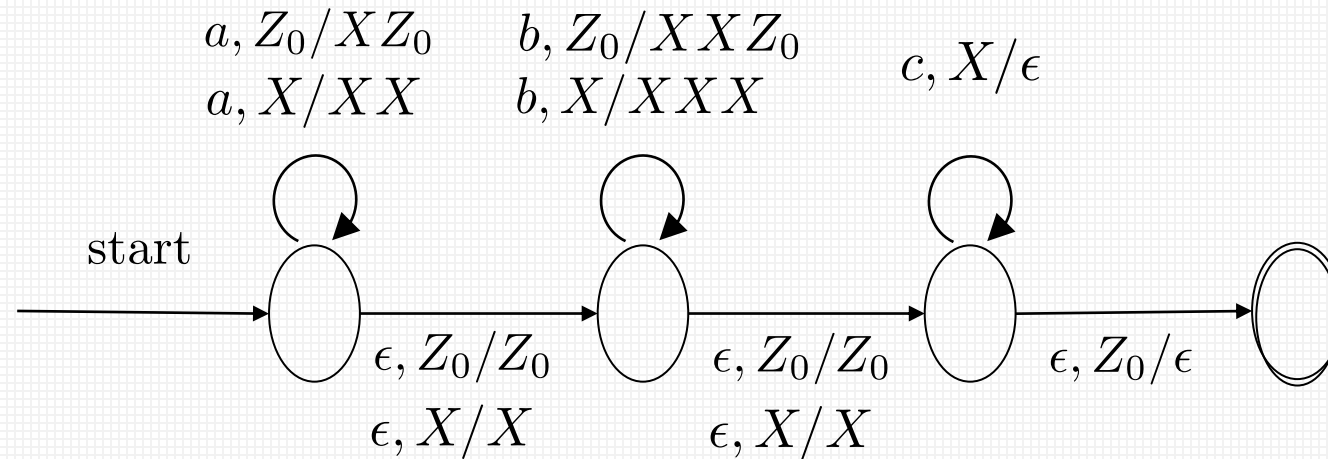
作答

下推自动机

例6：请构造下列语言的一个PDA（空栈接受或终态接受均可）

$$L = \{a^n b^m c^k \mid n \geq 0, m \geq 0, k \geq 0 \text{ 且 } n + 2m = k\}$$

解：构造终态接受的PDA。这道题的难点在于如何实现等式，其实只要让 a 、 b 和 c 的计数都使用 X 即可。



习题练习-上下文无关文法性质

例1： 用泵引理证明 $L = \{a^i b^j \mid j = i^2\}$ 不是上下文无关文法。

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法性质

例1： 用泵引理证明 $L = \{a^i b^j \mid j = i^2\}$ 不是上下文无关文法。

证明： 假设 L 是上下文无关的， 设 n 是由泵引理得到的常数。 考虑串 $z = a^n b^{n^2}$ 。 可以把 z 写作 $z = uvwxy$ ， 并且满足 $|vwx| \leq n$ 且 $vx \neq \epsilon$ 。

如果 vwx 只包含 a ， 由于 vx 不为空， 则 $uw^i y$ 中包含不到 n 个 a 以及 n^2 个 b ， 不属于 L ；

如果 vwx 只包含 b ， 由于 vx 不为空， 则 $uw^i y$ 中包含 n 个 a 以及不到 n^2 个 b ， 不属于 L ；

如果 vwx 既包含 a 也包含 b ， 分为三种情况：

1. v 包含 a 和 b ， x 为空或包含 b ： uv^2wx^2y 肯定不属于 L ， 因为可能会出现 a 和 b 交替出现的情况， 如 $aababb$ 。
2. v 为空或包含 a ， x 包含 a 和 b ： 和前面的情况类似。
3. v 包含 l 个 a ， x 包含 m 个 b ： $uv^{k+1}wx^{k+1}y$ 包含 $n+lk$ 个 a 和 n^2+mk 个 b ， 而 $(n+lk)^2 = n^2 + 2nkl + l^2k^2$ ， 则必须有 $m = 2nl + kl^2$ 成立， 这个条件不能保证永远满足， 不属于 L 。

习题练习-图灵机的设计

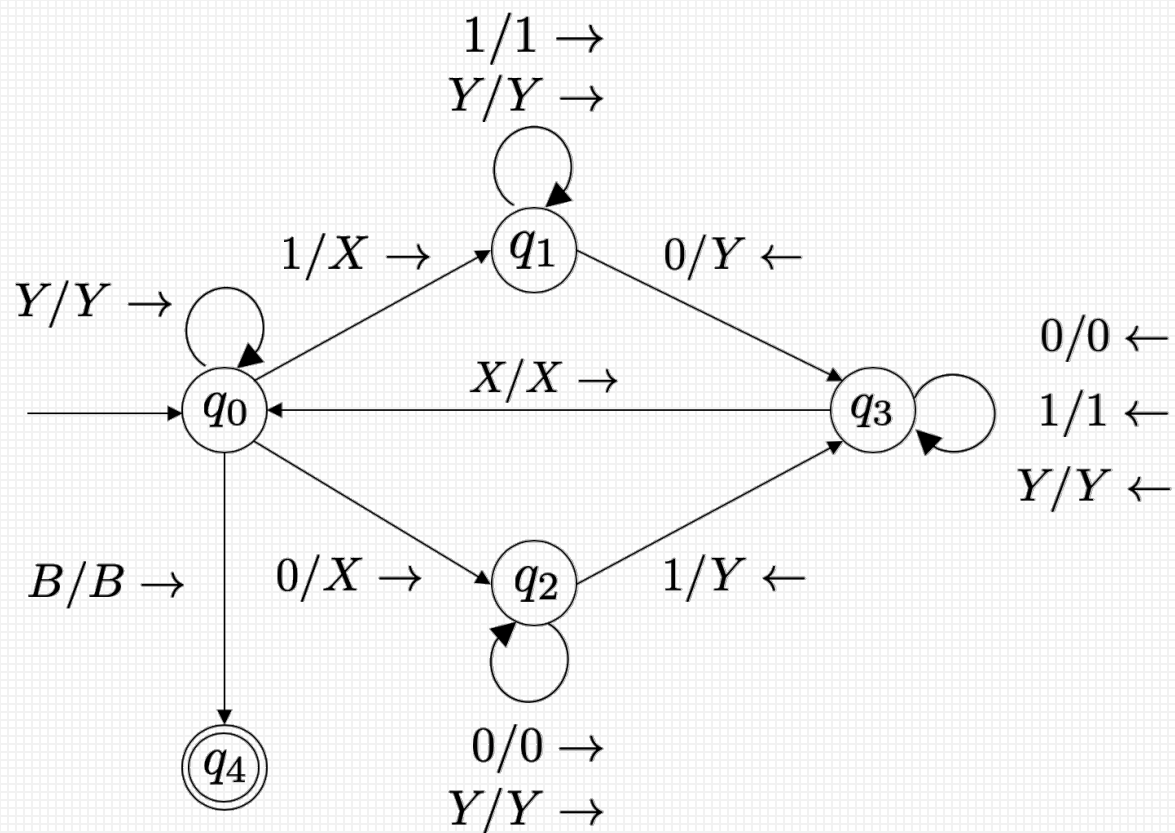
例1：为下列语言设计图灵机：带有相同个数的0和1的串的集合

正常使用主观题需2.0以上版本雨课堂

作答

上下文无关文法性质

例1: 为下列语言设计图灵机: 带有相同个数的0和1的串的集合



谢谢大家
祝大家考试顺利